

UJIAN AKHIR SEMESTER
BUSINESS INTELLIGENCE
PERANCANGAN *DATA WAREHOUSE* UNTUK ANALISIS PERILAKU
PENGGUNA SPOTIFY



Disusun Oleh:

2409116002	NURHIDAYAH
2409116007	JEN AGRESIA MISTI
2409116015	PUTRI SYAFANA AFRILLIA
2409116040	DHITA OLIVIA R. K.

SISTEM INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS MULAWARMAN
TAHUN 2025/2026

DAFTAR ISI

DAFTAR ISI	2
DAFTAR GAMBAR	4
DAFTAR <i>OUTPUT</i> PROGRAM	6
DAFTAR TABEL	7
BAB I PENDAHULUAN	8
1.1 Latar Belakang	8
1.2 Rumusan Masalah	9
1.3 Tujuan.....	9
1.4 Manfaat.....	9
BAB II TINJAUAN PUSTAKA	10
2.1 <i>Data warehouse</i>	10
2.2 <i>Extract, Transform, and Load (ETL)</i>	11
2.3 Google Colab.....	12
2.4 <i>Business intelligence</i>	12
2.5 <i>Churn</i>	12
2.6 SQLite	13
2.7 <i>Online Analytical Processing (OLAP)</i>	13
BAB III METODOLOGI	14
3.1 Sumber Data.....	14
3.1.1. Spotify <i>Churn</i> Analysis.....	14
3.1.2. Spotify SQL Exploration	15
3.1.3. <i>Dataset</i> Spotify Music	15
3.2 Arsitektur Sistem	16
3.3 Alur Perancangan	17
BAB IV IMPLEMENTASI ETL	19
4.1 Implementasi ETL.....	19
4.2 Hasil Akhir <i>Data warehouse</i>	38

BAB V HASIL ANALISIS.....	40
5.1 Analisis Data	40
5.1.1. Rata-Rata Popularitas Berdasarkan Genre	40
5.1.2. Tingkat <i>Churn</i> Berdasarkan Tipe Langganan	42
5.1.3. Perilaku Mendengarkan di 10 Negara Teratas	43
5.1.4. Hubungan Fitur Audio dengan <i>Skip Rate</i>	44
5.1.5. Tren Jumlah Pemutaran Lagu per Tahun	45
5.1.6. Korelasi <i>Churn</i> Pengguna dengan Fitur Audio	46
5.1.7. Preferensi Genre <i>Playlist</i> Berdasarkan Kelompok Usia	47
5.2 Visualisasi.....	48
5.2.1. <i>Dashboard</i>	49
5.2.2. Genre Teratas Berdasarkan Rata-Rata Popularitas	50
5.2.3. <i>Churn Rate</i> Berdasarkan Jenis Langganan	51
5.2.4. <i>Listening Behavior</i> Berdasarkan Negara	52
5.2.5. Trend <i>Plays</i> per Tahun	52
5.2.6. <i>Popularity Level</i> terhadap <i>Skip rate</i>	53
5.2.7. Perbandingan Pengguna Active dan <i>Churned</i>	54
5.2.8. <i>Playlist</i> Genre Berdasarkan Kelompok Umur	54
5.3 <i>Insight</i>	55
BAB VI PENUTUP	57
6.1 Kesimpulan.....	57
6.2 Saran.....	57
LAMPIRAN.....	58
DAFTAR PUSTAKA	70

DAFTAR GAMBAR

Gambar 2. 1 <i>Data warehouse</i>	10
Gambar 2. 2 <i>Star schema</i>	11
Gambar 2. 3 <i>Proses ETL</i>	11
Gambar 2. 4 <i>Google Colab</i>	12
Gambar 2. 5 <i>Dataset Churn Analysis</i>	Error! Bookmark not defined.
Gambar 2. 6 <i>Spotify SQL Exploration</i>	Error! Bookmark not defined.
Gambar 2. 7 <i>Dataset Spotify Music</i>	Error! Bookmark not defined.
Gambar 2. 8 <i>Activity Diagram</i>	Error! Bookmark not defined.
Gambar 3. 1 <i>Dataset Churn Analysis</i>	14
Gambar 3. 2 <i>Spotify SQL Exploration</i>	15
Gambar 3. 3 <i>Dataset Spotify Music</i>	16
Gambar 3. 4 <i>Activiy Diagram</i>	17
Gambar 4. 1 <i>Struktur database</i>	38
Gambar 4. 2 <i>Tabel di database</i>	39
Gambar 5. 1 <i>Key performance indicator</i>	49
Gambar 5. 2 <i>Dashboard (1)</i>	49
Gambar 5. 3 <i>Dashboard (2)</i>	50
Gambar 5. 4 <i>Genre Teratas</i>	50
Gambar 5. 5 <i>Churn Rate</i>	51
Gambar 5. 6 <i>Listening Behavior</i>	52
Gambar 5. 7 <i>Trend Plays</i>	53
Gambar 5. 8 <i>Popularity Level terhadap Skip rate</i>	53
Gambar 5. 9 <i>Perbandingan Pengguna Active dan Churned</i>	54
Gambar 5. 10 <i>Playlist Genre Berdasarkan Kelompok Umur</i>	55

DAFTAR KODE PROGRAM

Kode Program 1. Import <i>library</i>	19
Kode Program 2. Koneksi ke Drive	19
Kode Program 3. Pemanggilan <i>dataset</i>	20
Kode Program 4. Memeriksa <i>dataset</i>	21
Kode Program 5. <i>Transform Tracks</i>	23
Kode Program 6. Menyinkronkan tabel <i>Playlists</i>	24
Kode Program 7. Menyinkronkan tabel <i>Playlists</i>	26
Kode Program 8. Memperbaiki <i>overlap</i>	27
Kode Program 9. Buat <i>dim_track</i>	28
Kode Program 10. Buat <i>dim_playlist</i>	30
Kode Program 11. Buat <i>dim_user</i>	30
Kode Program 12. Pembuatan <i>dim_time</i>	31
Kode Program 13. Pembuatan tabel fakta.....	33
Kode Program 14. Penggabungan atribut	34
Kode Program 15. Pembuatan struktur <i>database</i>	35
Kode Program 16. Fungsi <i>run query</i>	40
Kode Program 17. <i>Top genres</i>	40
Kode Program 18. <i>Churn Rate by Subscription Type</i>	42
Kode Program 19. <i>Churn Rate by Subscription Type</i>	43
Kode Program 20. Audio dan <i>skip rate</i>	44
Kode Program 21. <i>Trend Plays per Year</i>	46
Kode Program 22. <i>Churn vs Audio features</i>	47
Kode Program 23. Genre <i>playlist</i>	48
Kode Program 24. <i>Dashboard Source Code</i>	58

DAFTAR *OUTPUT* PROGRAM

<i>Output Program 1.</i> Load Data	20
<i>Output Program 2.</i> <i>Quick Preview</i>	21
<i>Output Program 3.</i> Hasil <i>transform</i> tabel <i>Tracks</i>	23
<i>Output Program 4.</i> Hasil pembersihan <i>playlist</i>	25
<i>Output Program 5.</i> Hasil pembersihan <i>Users</i>	27
<i>Output Program 6.</i> Hasil cek <i>overlap</i>	28
<i>Output Program 7.</i> Pratinjau <i>dim_playlist</i>	30
<i>Output Program 8.</i> Hasil <i>dim_user</i>	30
<i>Output Program 9.</i> Hasil pembuatan dimensi waktu	32
<i>Output Program 10.</i> Preview hasil generate tabel <i>fact</i>	35
<i>Output Program 11.</i> Preview hasil generate tabel <i>fact</i>	37
<i>Output Program 12.</i> <i>Top genres</i>	41
<i>Output Program 13.</i> <i>Churn rate</i>	42
<i>Output Program 14.</i> <i>Listening Behavior</i>	44
<i>Output Program 15.</i> <i>Audio features</i> vs <i>Skip rate</i>	45
<i>Output Program 16.</i> <i>Trend Plays per Year</i>	46
<i>Output Program 17.</i> <i>Churn</i> vs <i>Audio features</i>	47
<i>Output Program 18.</i> Genre <i>playlist</i>	48

DAFTAR TABEL

Table 1. Hasil generate <i>database</i>	39
--	----

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi digital telah mendorong transformasi besar dalam industri hiburan, khususnya pada layanan *streaming* musik seperti Spotify. Platform ini memanfaatkan data dalam jumlah besar untuk memahami perilaku pengguna serta memberikan layanan yang lebih personal, seperti rekomendasi lagu berbasis histori. Hal ini menunjukkan bahwa data telah menjadi aset utama dalam mendukung pengalaman pengguna dan strategi bisnis digital (Khutami et al., 2024).

Tantangan utama yang dihadapi oleh *platform streaming* adalah mempertahankan pengguna agar tetap aktif karena jumlah pengguna yang sangat banyak. Fenomena pengguna yang berhenti menggunakan layanan atau yang dikenal sebagai *churn* menjadi permasalahan serius, karena berdampak langsung terhadap keberlanjutan laba. Biasanya industri berbasis *subscription*, biaya untuk memperoleh pelanggan baru dapat mencapai lima hingga tujuh kali lebih besar dibandingkan mempertahankan pelanggan yang sudah ada (Cheng & He, 2025). Diambil dari studi tersebut maka analisis terhadap perilaku pengguna menjadi sangat penting untuk mengidentifikasi pola yang berpotensi menyebabkan *churn*.

Diikuti masalah itu maka bidang *business intelligence* berperan sebagai pendekatan yang memungkinkan organisasi untuk mengolah data menjadi informasi yang bernilai dalam pengambilan keputusan. Salah satu komponen utamanya adalah *data warehouse* yang berfungsi sebagai tempat penyimpanan data terintegrasi yang telah melalui proses pembersihan dan transformasi sehingga siap untuk dianalisis. Banyak penelitian menunjukkan bahwa *usage data* dengan contoh frekuensi penggunaan, durasi aktivitas, atau interaksi antarmuka dengan sistem bisnis memiliki peran untuk memahami fenomena sebelumnya (Ahlstrand & Borg, 2025). Berarti, integrasi data yang baik melalui *data warehouse* dapat menjadi fondasi dalam menghasilkan *insight* yang akurat dan relevan.

Pada penelitian di laporan ini akan berfokus pada perancangan *data warehouse* untuk mendukung analisis perilaku pengguna dan *churn* pada *platform streaming* music Spotify.

Proses ini akan memanfaatkan proses *Extract, Transform, Load* (ETL) menggunakan Google Colab, data yang semula tersebar dan tidak terstruktur akan diolah menjadi informasi yang dapat untuk menghasilkan *insight* bisnis yang bermanfaat.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah dijelaskan, maka rumusan masalah dalam penelitian ini adalah sebagai berikut.

1. Bagaimana merancang *data warehouse* yang mampu mengintegrasikan data pengguna pada *platform* Spotify secara terstruktur?
2. Bagaimana mengimplementasikan proses *Extract, Transform, Load* (ETL) menggunakan Google Colab dalam pengolahan data pengguna?
3. Bagaimana memanfaatkan *data warehouse* untuk menganalisis perilaku pengguna berdasarkan *usage data* seperti durasi penggunaan, frekuensi akses, dan interaksi pengguna?
4. Bagaimana mengidentifikasi pola yang berpotensi menyebabkan *churn* pengguna berdasarkan hasil analisis data?

1.3 Tujuan

Tujuan dari penelitian ini adalah sebagai berikut:

1. Merancang *data warehouse* yang terstruktur untuk mengintegrasikan data pengguna pada *platform* Spotify.
2. Mengimplementasikan proses *Extract, Transform, Load* (ETL) menggunakan Google Colab dalam pengolahan data.
3. Menganalisis perilaku pengguna berdasarkan data penggunaan seperti durasi aktivitas, frekuensi akses, dan interaksi pengguna.
4. Mengidentifikasi pola yang berpotensi menyebabkan *churn* pengguna berdasarkan hasil analisis data.

1.4 Manfaat

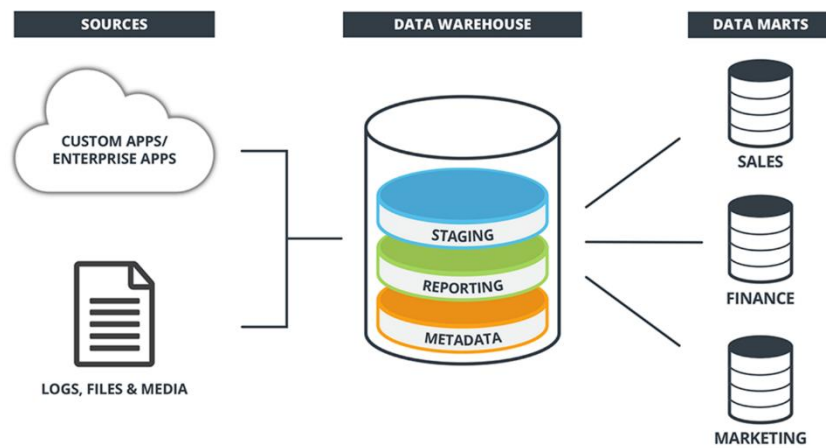
Penelitian ini diharapkan dapat memberikan pemahaman mengenai penerapan *data warehouse* dan *business intelligence* dalam menganalisis data pengguna pada *platform* digital terutama Spotify. Pengembangan ilmu pengetahuan selanjutnya diharapkan dapat melanjutkan studi terkait analisis perilaku pengguna dan *churn* pada layanan berbasis *subscription*, memberikan gambaran mengenai bagaimana *data warehouse* dapat dimanfaatkan untuk menghasilkan *insight* yang mendukung pengambilan keputusan bisnis, khususnya dalam memahami dan mengurangi tingkat *churn* pengguna.

BAB II

TINJAUAN PUSTAKA

2.1 Data warehouse

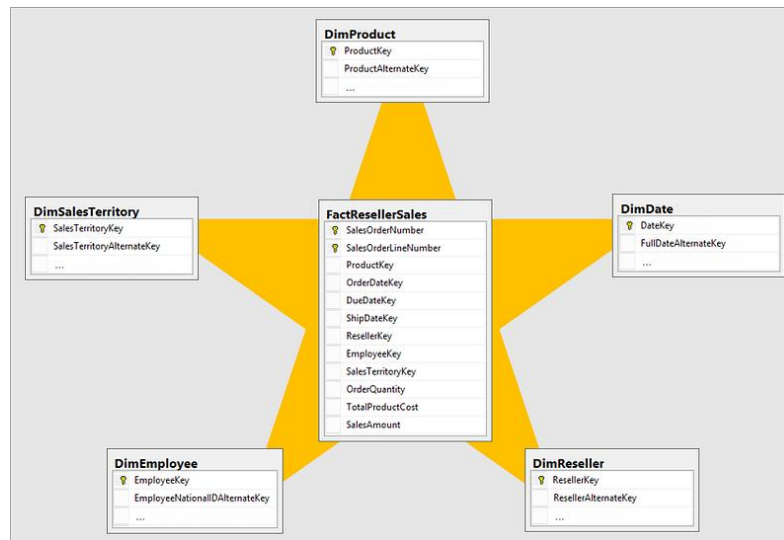
Data warehouse atau gudang data merupakan sistem penyimpanan data yang dirancang untuk mendukung proses analisis dan pengambilan keputusan. Berbeda dengan *database* operasional yang digunakan untuk transaksi harian, *data warehouse* berfungsi untuk mengintegrasikan data dari berbagai sumber yang telah melalui proses pembersihan dan transformasi sehingga menjadi lebih terstruktur dan konsisten untuk dianalisis.



Gambar 2. 1 Data warehouse

Data warehouse bersifat *subject-oriented*, *integrated*, *time-variant*, dan *non-volatile*, yang memungkinkan pengguna untuk melakukan analisis data historis secara efektif (Inmon, 2002). *Subject-oriented* menunjukkan bahwa data disusun berdasarkan subjek tertentu seperti pengguna atau transaksi. *Integrated* berarti data yang berasal dari berbagai sumber telah disatukan dalam format yang konsisten. *Time-variant* mengacu pada penyimpanan data historis dalam rentang waktu tertentu dan terakhir, *non-volatile* berarti data yang telah disimpan tidak mengalami perubahan atau penghapusan secara langsung.

Data warehouse umumnya menggunakan pendekatan pemodelan data seperti *star schema* yang terdiri dari tabel fakta (*fact table*) dan tabel dimensi (*dimension table*). Struktur ini memungkinkan proses analisis dilakukan dengan lebih efisien terlebih dalam pengolahan berskala besar (Kimball & Ross, 2002).

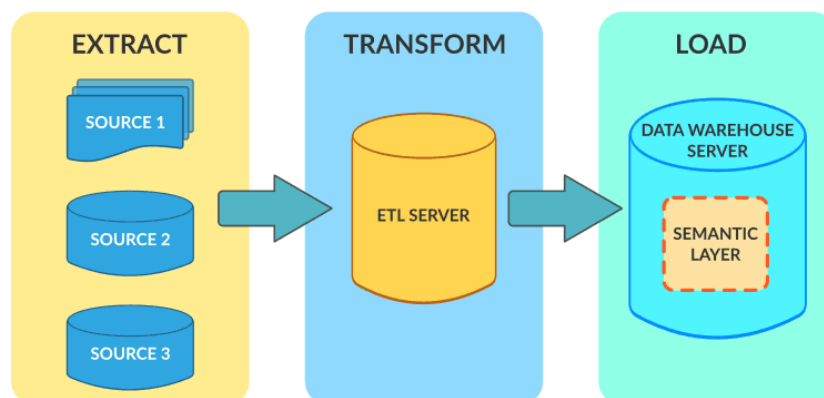


Gambar 2. 2 *Star schema*

Bidang *business intelligence* menjadikan *data warehouse* berperan sebagai fondasi utama dalam menyediakan data yang telah diolah sehingga dapat digunakan untuk menghasilkan informasi yang bernilai, lalu organisasi dapat melakukan perkembangan bisnis.

2.2 *Extract. Transform, and Load (ETL)*

Proses ETL (Extract, Transform, dan Load) merupakan langkah penting dalam pengembangan *data warehouse*. ETL bertujuan untuk mengambil data dari berbagai sumber, mengubah format data menjadi yang sesuai, dan memasukkan data ke dalam *data warehouse*. Proses ETL melibatkan beberapa tahap, yaitu ekstraksi (*extract*), transformasi (*transform*), dan pengisian (*load*).



Gambar 2. 3 Proses ETL

Ekstraksi melibatkan pengambilan data dari sumber data, transformasi melibatkan perubahan format data menjadi yang sesuai, dan pengisian melibatkan memasukkan data ke dalam *data warehouse*. ETL sangat penting dalam pengembangan *data warehouse* karena

dapat memastikan bahwa data yang diolah adalah akurat dan konsisten (Kimball & Ross, 2002).

2.3 Google Colab

Google Colab adalah platform gratis yang menyediakan lingkungan kerja Jupyter Notebook yang dapat diakses melalui *browser*. Platform ini dirancang untuk memudahkan pengguna dalam melakukan analisis data, pembelajaran mesin, dan pengembangan aplikasi.



Gambar 2. 4 Google Colab

Google Colab dapat digunakan sebagai alat bantu untuk memproses dan menganalisis data yang besar dan kompleks. Pengguna dapat mengakses berbagai *library* dan *framework* populer seperti Pandas, NumPy, dan scikit-learn untuk melakukan analisis data. Colab juga menyediakan kemampuan untuk menjalankan kode Python secara paralel, sehingga memungkinkan pengguna untuk memproses data yang besar dengan lebih cepat. Dengan demikian, Google Colab dapat menjadi alat bantu yang sangat berguna dalam proses *data warehouse*, terutama dalam tahap analisis dan visualisasi data.

2.4 Business intelligence

Business intelligence adalah bidang yang berfokus pada pengembangan sistem yang dapat membantu organisasi membuat keputusan yang lebih baik. BI melibatkan pengolahan data, analisis data, dan visualisasi data untuk membantu organisasi membuat keputusan yang lebih tepat. BI sangat penting dalam pengembangan *data warehouse* karena dapat membantu organisasi membuat keputusan yang lebih baik dan meningkatkan efisiensi bisnis (Turban et al., 2017).

2.5 Churn

Churn adalah istilah yang digunakan untuk menggambarkan perubahan dalam data yang terjadi karena beberapa alasan, seperti perubahan format data atau perubahan sumber data.

Churn dapat menyebabkan kesalahan dalam pengolahan data dan dapat mempengaruhi kualitas data. Oleh karena itu, penting untuk mengidentifikasi dan mengatasi *churn* dalam pengembangan *data warehouse* (Kimball & Ross, 2002).

2.6 SQLite

SQLite adalah sebuah pustaka perangkat lunak bahasa C yang mengimplementasikan mesin *database* SQL yang bersifat *self-contained*, tanpa server atau *serverless*, tanpa, dan transaksional. Karakteristik utama SQLite yang menyimpan seluruh data relasional dalam satu *file* teks biasa di perangkat menjadikannya sangat efisien untuk digunakan sebagai *prototyping* pengolahan data berskala kecil hingga menengah tanpa memerlukan instalasi *server database* eksternal yang kompleks (SQLite Home Page, 2025).

2.7 Online Analytical Processing (OLAP)

Online Analytical Processing (OLAP) adalah sebuah teknologi perangkat lunak yang dirancang khusus untuk melakukan analisis, pelaporan, dan sintesis data historis berukuran besar secara dinamis dan interaktif dari berbagai perspektif multidimensi. Berbeda dengan sistem operasional harian atau *Online Transaction Processing* (OLTP) yang berfokus pada kecepatan pemrosesan transaksi baris per baris (*insert/update*), OLAP dioptimalkan untuk mengeksekusi *query* yang kompleks serta agregasi data tingkat tinggi guna mendukung proses pengambilan keputusan strategis (*Decision Support System*). Menurut Kimball (2013).

OLAP merupakan suatu model dimensional yang diimplementasikan pada lingkungan *database* multidimensional untuk memfasilitasi analisis bisnis. Sementara itu, Connolly dan Begg (2005) mendefinisikan OLAP sebagai sebuah sintesis dinamis, analisis, dan gabungan dari data multidimensional berskala besar yang mampu menyajikan informasi terintegrasi secara cepat. Dengan demikian, OLAP bertindak sebagai jembatan yang mengubah data mentah di dalam *data warehouse* menjadi *insight* atau wawasan bisnis yang siap pakai bagi pihak eksekutif.

BAB III

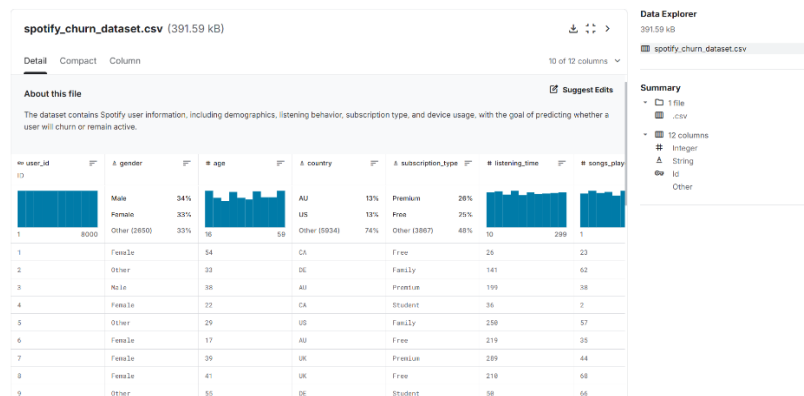
METODOLOGI

3.1 Sumber Data

Sumber data yang digunakan dalam penelitian ini berasal dari beberapa *dataset* publik Spotify yang diperoleh melalui platform Kaggle. *Dataset* tersebut digunakan untuk mendukung proses analisis perilaku pengguna serta perancangan *data warehouse*. Seluruh data memiliki format CSV dan diolah menggunakan Google Colab. *Dataset* yang digunakan terdiri dari data pengguna, data lagu, dan data *playlist*. Ketiga *dataset* tersebut saling melengkapi dan digunakan untuk membangun tabel fakta dan tabel dimensi dalam *data warehouse*.

3.1.1. Spotify Churn Analysis

Dataset utama berasal dari Spotify *Dataset* for Churn Analysis (<https://www.kaggle.com/datasets/nabihazahid/spotify-dataset-for-churn-analysis>). *Dataset* ini berisi data pengguna yang berkaitan dengan aktivitas penggunaan serta status *churn*.

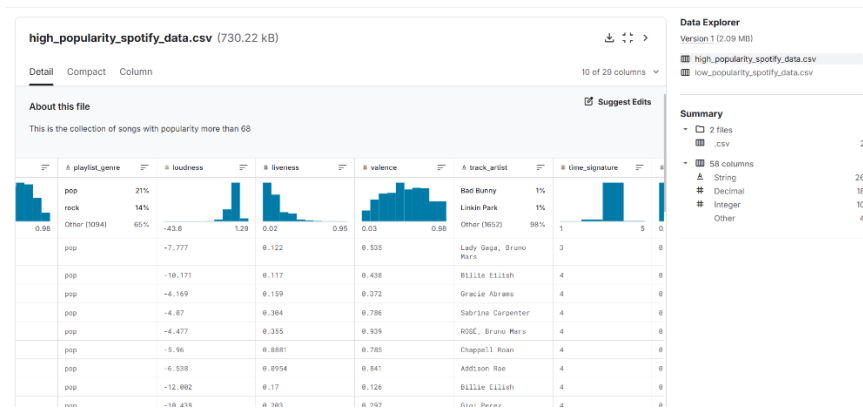


Gambar 3. 1 Dataset Churn Analysis

Atribut dalam *dataset* meliputi *user_id*, usia, jenis kelamin, negara, jenis langganan, durasi mendengarkan, jumlah lagu yang diputar per hari, tingkat skip, penggunaan fitur *offline*, jumlah iklan yang didengarkan, serta status *churn*. Data ini menggambarkan perilaku pengguna dalam menggunakan layanan Spotify dalam periode tertentu.

Data bersifat *user-centric* karena berfokus pada aktivitas pengguna. Nilai-nilai yang tersedia dapat digunakan sebagai indikator dalam analisis perilaku, seperti intensitas penggunaan dan interaksi dengan fitur *platform*. *Dataset* ini digunakan sebagai dasar

Dataset ketiga berasal dari *Spotify Music Dataset* (<https://www.kaggle.com/datasets/solomonameh/spotify-music-dataset>). *Dataset* ini digunakan untuk melengkapi informasi terkait lagu yang tidak sepenuhnya tersedia pada *dataset* sebelumnya. Atribut yang tersedia meliputi *track_id*, nama lagu, artis, album, genre, popularitas, durasi lagu, dan status eksplisit. Data ini memberikan informasi tambahan yang dapat digunakan untuk memperkaya analisis terhadap karakteristik musik.



Gambar 3. 3 Dataset Spotify Music

Data bersifat atributif dan berfungsi sebagai referensi tambahan dalam *data warehouse*. Dataset ini digunakan untuk memperkuat dimensi *track*, sehingga analisis dapat dilakukan berdasarkan berbagai aspek seperti genre, popularitas, dan durasi lagu.

3.2 Arsitektur Sistem

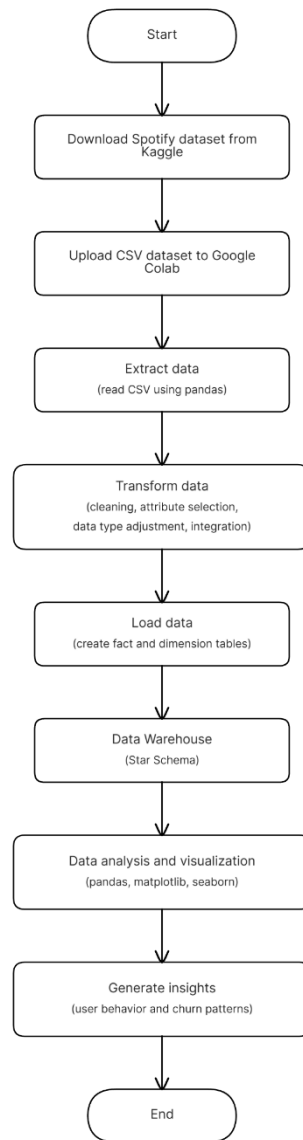
Arsitektur sistem dalam penelitian ini dirancang untuk mendukung proses pengolahan data hingga menghasilkan informasi yang dapat digunakan dalam analisis perilaku pengguna Spotify. Sistem terdiri dari beberapa komponen utama yaitu sumber data, proses ETL, penyimpanan data, dan analisis data.

Sumber data berasal dari beberapa *dataset* Spotify dalam format CSV yang diunduh dari Kaggle. Data tersebut dimuat ke dalam Google Colab sebagai lingkungan pemrosesan utama. Proses ETL dilakukan menggunakan Python di Google Colab. Tahap *extract* dilakukan dengan membaca *file* CSV menggunakan *library* pandas. Data kemudian masuk ke tahap *transform*, yang meliputi pembersihan data, penghapusan duplikat, penanganan nilai kosong, serta penyesuaian tipe data. Selain itu dilakukan penggabungan data antar *dataset* menggunakan atribut kunci seperti *track_id*.

Hasil transformasi disusun dalam bentuk struktur *data warehouse*. Struktur yang digunakan mengacu pada pendekatan *star schema* yang terdiri dari tabel fakta dan tabel dimensi. Tabel fakta menyimpan data aktivitas pengguna seperti durasi mendengarkan dan frekuensi penggunaan. Tabel dimensi menyimpan informasi pendukung seperti data pengguna, lagu, dan *playlist*. Data yang telah terstruktur kemudian digunakan untuk proses analisis. Analisis dilakukan di Google Colab menggunakan *library* seperti pandas, matplotlib, dan seaborn. Hasil analisis ditampilkan dalam bentuk visualisasi untuk mendukung interpretasi data.

3.3 Alur Perancangan

Alur perancangan sistem dalam penelitian ini disusun untuk menggambarkan tahapan pengolahan data mulai dari pengambilan *dataset* hingga menghasilkan *insight*. Alur tersebut divisualisasikan dalam bentuk UML Activity Diagram yang menunjukkan urutan proses secara terstruktur.



Gambar 3. 4 *Activiy Diagram*

Proses dimulai dari tahap pengambilan data dengan mengunduh *dataset* Spotify dari platform Kaggle. *Dataset* yang telah diunduh kemudian diunggah ke dalam Google Colab untuk diproses lebih lanjut. Tahap berikutnya adalah proses *extract*, yaitu membaca file CSV menggunakan *library pandas*. Data yang telah dimuat kemudian masuk ke tahap *transform*. Pada tahap ini dilakukan pembersihan data, seleksi atribut yang relevan, penyesuaian tipe data, serta integrasi beberapa *dataset* menjadi satu kesatuan data yang terstruktur.

Setelah proses transformasi selesai, data masuk ke tahap *load*. Pada tahap ini dilakukan pembentukan tabel fakta dan tabel dimensi sesuai dengan kebutuhan *data warehouse*. Tabel fakta digunakan untuk menyimpan data aktivitas pengguna, sedangkan tabel dimensi menyimpan informasi pendukung seperti data pengguna dan data lagu. Data yang telah disusun kemudian membentuk *data warehouse* dengan pendekatan *star schema*. Struktur ini memungkinkan proses analisis dilakukan dengan lebih efisien karena data telah terorganisir dengan baik.

Tahap selanjutnya adalah analisis dan visualisasi data. Proses ini dilakukan menggunakan Google Colab dengan bantuan *library* seperti *pandas*, *matplotlib*, dan *seaborn*. Hasil analisis digunakan untuk mengidentifikasi pola perilaku pengguna berdasarkan aktivitas penggunaan. Tahap akhir dari alur ini adalah menghasilkan *insight*, terutama yang berkaitan dengan perilaku pengguna dan pola *churn*. *Insight* yang dihasilkan dapat digunakan sebagai dasar dalam pengambilan keputusan dan pengembangan strategi bisnis.

BAB IV

IMPLEMENTASI ETL

4.1 Implementasi ETL

Proses ETL dalam penelitian ini dilakukan menggunakan Google Colab sebagai lingkungan pengolahan data. Tahapan ETL terdiri dari tiga proses utama, yaitu *extract*, *transform*, dan *load*. Proses *extract* dilakukan dengan mengambil data dari beberapa *dataset* Spotify dalam format CSV. Data tersebut kemudian diproses pada tahap *transform* untuk membersihkan, menyesuaikan, dan mengintegrasikan data agar sesuai dengan kebutuhan analisis. Tahap *load* dilakukan dengan menyusun data ke dalam struktur *data warehouse* yang terdiri dari tabel fakta dan tabel dimensi.

4.1.1. Extract Data

Tahap *extract* dilakukan untuk mengambil data dari sumber *dataset* dan memuatnya ke dalam lingkungan Google Colab. *Dataset* yang digunakan terdiri dari data lagu (*tracks*), data *playlist*, dan data pengguna (*users*) dalam format CSV. Proses diawali dengan melakukan *import library* yang dibutuhkan untuk pengolahan data.

Kode Program 1. *Import library*

```
1. import pandas as pd
2. import numpy as np
3. import sqlite3
4. import os
5. import matplotlib.pyplot as plt
6. import seaborn as sns
7. from IPython.display import display
```

Library pandas digunakan untuk membaca dan memanipulasi data, *numpy* untuk operasi numerik, *sqlite3* untuk koneksi *database*, serta *os* untuk pengelolaan *path file*. Selanjutnya dilakukan koneksi Google Colab dengan Google Drive agar *dataset* dapat diakses. Setelah Google Drive terhubung, ditentukan direktori penyimpanan *dataset*. *Dataset* kemudian dibaca menggunakan fungsi `read_csv()` dan disimpan ke dalam *dataframe* yang berbeda.

Kode Program 2. Koneksi ke *Drive*

```
8. from google.colab import drive
9. drive.mount('/content/drive')
10.
```

```
11. PATH = "/content/drive/MyDrive/dataset BIawak/"
12. DB_PATH = os.path.join(PATH, "spotify_dw.db")
```

Output Program 1. Load Data

```
13. Drive already mounted at /content/drive; to attempt to forcibly remount, call
    drive.mount("/content/drive", force_remount=True).
14. =====
15. EXTRACT - RAW SHAPE
16. =====
17. df_tracks   : (6300, 8)
18. df_playlist : (1686, 29)
19. df_users    : (8000, 12)
```

Data yang berhasil dimuat selanjutnya diperiksa ukuran dimensinya untuk memastikan proses *extract* berjalan dengan baik. Hasilnya menunjukkan bahwa *dataset tracks* memiliki 6300 baris dan 8 kolom, *dataset playlist* memiliki 1686 baris dan 29 kolom, serta *dataset users* memiliki 8000 baris dan 12 kolom.

Bagian Load *Dataset* atau Extract ini adalah langkah awal sebelum kita melakukan analisis data. Di sini, kita perlu mengimport beberapa *library* yang penting seperti *pandas* dan *numpy* untuk mengolah data, *sqlite3* untuk menghubungkan dengan *database*, serta *matplotlib* dan *seaborn* untuk membuat visualisasi data. Kita juga perlu memasang Google Drive agar kita bisa mengakses file *dataset* yang disimpan secara online.

Langkah selanjutnya adalah menentukan lokasi direktori utama yang mengarah ke folder *dataset* di Google Drive. Setelah itu, kita buat lokasi khusus untuk masing-masing file, yaitu data lagu, *playlist*, dan pengguna. Ketiga *dataset* ini kemudian kita baca menggunakan fungsi *read_csv()* dan disimpan ke dalam *DataFrame* terpisah, yaitu *df_tracks*, *df_playlist*, dan *df_users*. Sebagai langkah untuk memastikan bahwa data kita sudah benar, kita tampilkan ukuran dari masing-masing *dataset*. Hasilnya menunjukkan bahwa *dataset* lagu memiliki 6300 baris dan 8 kolom, *playlist* memiliki 1686 baris dan 29 kolom, serta pengguna memiliki 8000 baris dan 12 kolom. Langkah ini sangat penting untuk memastikan tidak ada kesalahan dalam proses *extract* sebelum kita melanjutkan ke tahap transformasi dan pemodelan *data warehouse*.

Kode Program 3. Pemanggilan dataset

```
20. df_tracks   = pd.read_csv(path + "spotify_tracks.csv")
21. df_playlist = pd.read_csv(path + "high_popularity_spotify_data.csv")
22. df_users    = pd.read_csv(path + "spotify_churn_dataset.csv")
23.
24. print(f"df_tracks   : {df_tracks.shape}")
```

```
25. print(f"df_playlist : {df_playlist.shape}")
26. print(f"df_users      : {df_users.shape}")
```

Setelah itu dilakukan pemeriksaan awal terhadap struktur data, meliputi nama kolom, nilai kosong (*missing values*), dan data duplikat. Bagian Load *Dataset* (Extract) ini digunakan untuk mempersiapkan data sebelum masuk ke tahap analisis. Pertama, dilakukan import berbagai *library* yang dibutuhkan seperti pandas dan numpy untuk pengolahan data, sqlite3 untuk koneksi *database*, serta matplotlib dan seaborn untuk visualisasi. Google Drive di-mount agar notebook dapat mengakses file *dataset* yang tersimpan secara online.

Selanjutnya, ditentukan path direktori utama yang mengarah ke folder *dataset* di Google Drive, kemudian dibuat path khusus untuk masing-masing file, yaitu data *tracks*, *playlist*, dan *users*. Ketiga *dataset* tersebut kemudian dibaca menggunakan fungsi `read_csv()` dan disimpan ke dalam DataFrame terpisah, yaitu `df_tracks`, `df_playlist`, dan `df_users`.

Kode Program 4. Memeriksa *dataset*

```
27. for name, df in [("TRACKS", df_tracks), ("PLAYLIST", df_playlist), ("USERS", df_users)]:
28.     print(f"\n{'='*70}")
29.     print(f"[{name}] Shape: {df.shape}")
30.     print(f"[{name}] Columns ({len(df.columns)}):")
31.     print(list(df.columns))
32.     print(f"[{name}] Missing values:")
33.     missing = df.isnull().sum()
34.     display(missing[missing > 0] if (missing > 0).any() else pd.Series(dtype='int64'))
35.     print(f"[{name}] Duplicates: {df.duplicated().sum()}")
36.     print(f"[{name}] Head:")
37.     display(df.head(3))
```

Sebagai langkah validasi awal, ditampilkan ukuran (*shape*) dari masing-masing *dataset* untuk memastikan bahwa data berhasil dimuat dengan benar. Hasilnya menunjukkan bahwa *dataset tracks* memiliki 6300 baris dan 8 kolom, *playlist* memiliki 1686 baris dan 29 kolom, serta *users* memiliki 8000 baris dan 12 kolom. Tahap ini penting untuk memastikan tidak ada kesalahan dalam proses extract sebelum melanjutkan ke tahap transformasi dan pemodelan *data warehouse*.

Output Program 2. Quick Preview

```
38. [TRACKS] Shape: (6300, 8)
39. [TRACKS] Columns (8):
40. ['id', 'name', 'genre', 'artists', 'album', 'popularity', 'duration_ms', 'explicit']
41. [TRACKS] Missing values: 0
42.
43. dtype: int64
44. [TRACKS] Duplicates: 0
45. [TRACKS] Head:
```

```
46. [PLAYLIST] Shape: (1686, 29)
47. [PLAYLIST] Columns (29):
48. ['energy', 'tempo', 'danceability', 'playlist_genre', 'loudness', 'liveness',
    'valence', 'track_artist', 'time_signature', 'speechiness', 'track_popularity',
    'track_href', 'uri', 'track_album_name', 'playlist_name', 'analysis_url', 'track_id',
    'track_name', 'track_album_release_date', 'instrumentalness', 'track_album_id', 'mode',
    'key', 'duration_ms', 'acousticness', 'id', 'playlist_subgenre', 'type', 'playlist_id']
49. [PLAYLIST] Missing values: 0
50.
51. [USERS] Shape: (8000, 12)
52. [USERS] Columns (12):
53. ['user_id', 'gender', 'age', 'country', 'subscription_type', 'listening_time',
    'songs_played_per_day', 'skip_rate', 'device_type', 'ads_listened_per_week',
    'offline_listening', 'is_churned']
54. [USERS] Missing values: 0
55.
56. dtype: int64
57. [USERS] Duplicates: 0
58. [USERS] Head:
```

Bagian Quick Preview di atas ini digunakan untuk melihat gambaran awal dari *dataset tracks*, *playlist*, dan *users*. Pengecekan yang dilakukan meliputi jumlah data (shape), nama kolom, nilai yang hilang (missing values), serta data duplikat. Hasilnya, *dataset tracks* (6300 baris, 8 kolom) dan *users* (8000 baris, 12 kolom) tidak memiliki missing values maupun duplikasi, sehingga sudah cukup bersih. Sementara *dataset playlist* (1686 baris, 29 kolom) memiliki sedikit missing values pada kolom *track_album_name*, namun tidak terdapat duplikasi data. Ditampilkan beberapa baris awal dari masing-masing *dataset* untuk memahami isi data secara cepat. Tahap ini penting untuk memastikan kualitas data sebelum masuk ke proses transformasi selanjutnya.

4.1.2. Transform Data

Transformasi data adalah proses mengubah format, struktur, atau nilai data mentah agar lebih sesuai untuk kebutuhan sistem, analisis statistik, atau pemodelan. proses transformasi data akan berfokus pada penyesuaian struktur dan pembersihan minor agar seluruh *dataset* siap digunakan untuk analisis atau pemodelan. Mengingat *dataset tracks* dan *users* sudah bersih dari data duplikat maupun nilai yang hilang, transformasi pada kedua *dataset* ini akan lebih diarahkan pada standardisasi format kolom.

4.1.2.1. Data Tracks

Bagian Transform di bagian *Tracks* ini bertujuan untuk membersihkan dan menyesuaikan struktur data pada *dataset tracks* agar lebih konsisten dan siap digunakan dalam *data warehouse*. Proses dimulai dengan menyalin *dataset* asli, kemudian dilakukan penggantian nama kolom seperti id menjadi *track_id*, name menjadi *track_name*, dan lainnya agar lebih jelas dan terstandarisasi.

Kode Program 5. Transform Tracks

```
59. df_tracks_clean = df_tracks.copy()
60.
61. df_tracks_clean = df_tracks_clean.rename(columns={
62.     "id": "track_id",
63.     "name": "track_name",
64.     "artists": "artist_name",
65.     "album": "album_name",
66.     "popularity": "track_popularity",
67. })
68.
69. df_tracks_clean["explicit"] = df_tracks_clean["explicit"].astype(bool)
70. df_tracks_clean["track_popularity"] =
    pd.to_numeric(df_tracks_clean["track_popularity"],
    errors="coerce").fillna(0).astype(int)
71. df_tracks_clean["duration_ms"] = pd.to_numeric(df_tracks_clean["duration_ms"],
    errors="coerce").fillna(0).astype(int)
72.
73. df_tracks_clean["duration_min"] = (df_tracks_clean["duration_ms"] / 60000).round(2)
74. df_tracks_clean["popularity_level"] = pd.cut(
75.     df_tracks_clean["track_popularity"],
76.     bins=[-1, 30, 70, 100],
77.     labels=["Low", "Medium", "High"]
78. )
79.
80. print("df_tracks_clean:", df_tracks_clean.shape)
81. display(df_tracks_clean[["track_id", "track_name", "artist_name", "track_popularity",
    "duration_min", "popularity_level"]].head())
```

Output Program 3. Hasil transform tabel Tracks

```

82. df_tracks_clean: (6300, 10)
83.
84. track_id      track_name      artist_name      track_popularity      duration_min
      popularity_level
85. 7kr3xZk4yb3YSZ4VFtg2Qt Acoustic      Billy Raffoul      58      2.87      Medium
86. 1kJygfS4eoVziBBI93MSYp Acoustic      Billy Raffoul      57      2.87      Medium
87. 6lynns69p4zTCRxmmsY1x Here Comes the Sun - Acoustic Molly Hocking, Bailey Rushlow 42
      2.41      Medium
88. 1RC9slv335IfLce5vt9KTW Acoustic #3      The Goo Goo Dolls      46      1.94      Medium
89. 5o9L8xBuILoVjLECSBi7Vo My Love Mine All Mine - Acoustic Instrumental      Guus
      Dielissen, Casper Esmann      33      2.23      Medium

```

Selanjutnya, dilakukan penyesuaian tipe data (casting), seperti mengubah kolom explicit menjadi boolean serta memastikan *track_popularity* dan *duration_ms* bertipe numerik. Selain itu, ditambahkan atribut turunan yaitu *duration_min* untuk mengubah durasi lagu ke satuan menit, serta *popularity_level* yang mengelompokkan popularitas lagu menjadi kategori Low, Medium, dan High. Hasil akhirnya menunjukkan *dataset* yang sudah lebih rapi dengan total 6300 baris dan 10 kolom, serta siap digunakan untuk tahap pembentukan tabel dimensi pada *data warehouse*.

4.1.2.2. Data Playlist

Tujuan di bagian ini adalah untuk membersihkan dan menyeragamkan struktur data pada *dataset playlist*. Proses dimulai dengan menangani nilai yang hilang, seperti mengisi kolom *track_album_name* dengan nilai default agar tidak kosong. Selanjutnya dilakukan penyesuaian nama kolom agar konsisten dengan *dataset* lain, misalnya *track_id*, *artist_name*, dan *album_name*.

Kode Program 6. Menyinkronkan tabel *Playlists*

```

90. df_playlist_clean = df_playlist.copy()
91.
92. df_playlist_clean["track_album_name"] =
      df_playlist_clean["track_album_name"].fillna("Unknown Album")
93.
94. df_playlist_clean = df_playlist_clean.rename(columns={
95.     "id": "playlist_row_id",
96.     "track_artist": "artist_name",
97.     "track_album_name": "album_name",
98. })
99.
100.     playlist_cols_keep = [
101.         "track_id", "track_name", "artist_name", "album_name",
102.         "track_album_release_date", "track_album_id", "track_popularity",
103.         "playlist_id", "playlist_name", "playlist_genre", "playlist_subgenre",
104.         "energy", "tempo", "danceability", "loudness", "liveness", "valence",

```



```

105.         "acousticness", "instrumentalness", "speechiness", "key", "mode",
106.         "time_signature", "duration_ms"
107.     ]
108.     df_playlist_clean = df_playlist_clean[playlist_cols_keep].copy()
109.
110.     df_playlist_clean["track_album_release_date"] = pd.to_datetime(
111.         df_playlist_clean["track_album_release_date"], errors="coerce"
112.     )
113.
114.     audio_cols = [
115.         "energy", "tempo", "danceability", "loudness", "liveness", "valence",
116.         "acousticness", "instrumentalness", "speechiness", "key", "mode",
117.         "time_signature", "track_popularity", "duration_ms"
118.     ]
119.     for col in audio_cols:
120.         df_playlist_clean[col] = pd.to_numeric(df_playlist_clean[col],
121.         errors="coerce")
122.
123.     df_playlist_clean["duration_ms"] = df_playlist_clean["duration_ms"].fillna(0)
124.     df_playlist_clean["duration_min"] = (df_playlist_clean["duration_ms"] /
125.     60000).round(2)
126.
127.     print("df_playlist_clean:", df_playlist_clean.shape)
128.     display(df_playlist_clean.head())

```

Hanya kolom-kolom yang relevan dipilih untuk digunakan dalam analisis, terutama yang berkaitan dengan informasi lagu dan fitur audio. Tipe data juga disesuaikan, seperti mengubah kolom tanggal menjadi format datetime serta memastikan atribut numerik (misalnya tempo, danceability, dll.) bertipe angka. Ditambahkan atribut turunan berupa `duration_min` untuk mengubah durasi lagu ke satuan menit.

Output Program 4. Hasil pembersihan `playlist`

```

127. df_playlist_clean: (1686, 25)
128.      track_id      track_name      artist_name      album_name
      track_album_release_date      track_album_id track_popularity      playlist_id
      playlist_name playlist_genre ...      liveness      valence acousticness
      instrumentalness      speechiness      key      mode      time_signature duration_ms
      duration_min
129.      0      2plbrEY59IikOBgBGLjaoe Die With A Smile      Lady Gaga, Bruno Mars Die
      With A Smile      2024-08-16      10FLjwfbpxLmW8c25Xyc2N 100      37i9dQZF1DXcBWIGoYBM5M
      Today's Top Hits      pop      ...      0.122      0.535      0.3080      0.0000      0.0304      6
      0      3      251668      4.19
130.      1      6dOtVTDDiaauQNBQEDotLAB BIRDS OF A FEATHER      Billie Eilish      HIT ME HARD
      AND SOFT      2024-05-17      7aJuG4TFXa2hmE4z1yxc3n 97      37i9dQZF1DXcBWIGoYBM5M
      Today's Top Hits      pop      ...      0.117      0.438      0.2000      0.0608      0.0358      2
      1      4      210373      3.51

```

131.	2	7ne4VBA60CxGM75vw0EYad That's So True Gracie Abrams The Secret of Us	(Deluxe)	2024-10-18	0hBRqPYPXhr1RkTDG3n4Mk 93	37i9dQZF1DXcBWIGoYBM5M	Today's Top Hits	pop	...	0.159	0.372	0.2140	0.0000	0.0368	1
	1	4	166300	2.77											
132.	3	1d7PtW3qYcfpdLNL5REhtJ Taste Sabrina Carpenter Short n' Sweet		2024-08-23	4B4Elma4nNDUy16D5PvQkj 81	37i9dQZF1DXcBWIGoYBM5M	Today's Top Hits	pop	...	0.304	0.786	0.0939	0.0000	0.0634	0
			157280	2.62											4
133.	4	5vNRhkKd0yEAg8suGBpjeY APT. ROSÉ, Bruno Mars APT. 2024-10-18			2IYQwWgXgOIIn7t3iF6ufFD 98	37i9dQZF1DXcBWIGoYBM5M	Today's Top Hits	pop	...	0.355	0.939	0.0283	0.0000	0.2600	0
														4	169917
			2.83												

Hasil akhirnya adalah *dataset playlist* yang sudah lebih bersih, konsisten, dan siap digunakan dalam proses pembentukan tabel dimensi dan analisis *data warehouse*.

4.1.2.3. Data Users

Bagian ini dilakukan untuk menyesuaikan tipe data dan menambahkan atribut turunan pada *dataset users*. Proses dimulai dengan mengubah beberapa kolom ke tipe yang sesuai, seperti *is_churned* dan *offline_listening* menjadi boolean, serta memastikan kolom numerik seperti *skip_rate*, *listening_time*, *songs_played_per_day*, dan *ads_listened_per_week* bertipe angka.

Kode Program 7. Menyinkronkan tabel *Playlists*

```

134. df_users_clean = df_users.copy()
135.
136. df_users_clean["is_churned"] = df_users_clean["is_churned"].astype(bool)
137. df_users_clean["offline_listening"] = df_users_clean["offline_listening"].astype(bool)
138. df_users_clean["skip_rate"] = pd.to_numeric(df_users_clean["skip_rate"],
errors="coerce").fillna(0).round(4)
139. df_users_clean["listening_time"] = pd.to_numeric(df_users_clean["listening_time"],
errors="coerce").fillna(0).round(2)
140. df_users_clean["songs_played_per_day"] = pd.to_numeric(df_users_clean["songs_played_per_day"], errors="coerce").fillna(0)
141. df_users_clean["ads_listened_per_week"] = pd.to_numeric(df_users_clean["ads_listened_per_week"], errors="coerce").fillna(0)
142.
143. df_users_clean["age_group"] = pd.cut(
144.     df_users_clean["age"],
145.     bins=[0, 18, 25, 35, 50, 100],
146.     labels=["<18", "18-25", "26-35", "36-50", "50+"]
147. )
148.
149. df_users_clean["engagement_level"] = pd.cut(
150.     df_users_clean["listening_time"],
151.     bins=[-1, 120, 240, 10000],

```

```

152.     labels=["Low", "Medium", "High"]
153. )
154.
155. print("df_users_clean:", df_users_clean.shape)
156. display(df_users_clean[[
157.     "user_id", "gender", "age", "age_group", "country",
158.     "subscription_type", "listening_time", "skip_rate",
159.     "engagement_level", "is_churned"
160. ]].head())

```

Dibuat atribut turunan untuk memperkaya analisis. Kolom *age_group* digunakan untuk mengelompokkan pengguna berdasarkan rentang usia, sedangkan *engagement_level* dibuat dari *listening_time* untuk mengklasifikasikan tingkat aktivitas pengguna menjadi Low, Medium, dan High. Hasilnya adalah *dataset users* yang lebih terstruktur dengan 8000 baris dan 14 kolom, serta siap digunakan dalam analisis lanjutan dan pembentukan tabel dimensi pada *data warehouse*.

Output Program 5. Hasil pembersihan *Users*

```

161. df_users_clean: (8000, 14)
162. user_id gender age age_group country subscription_type listening_time
      skip_rate engagement_level is_churned
163. 0      1   Female 54      50+   CA      Free      26      0.20   Low   True
164. 1      2   Other 33     26-35  DE      Family 141      0.34   Medium Fals
165. 2      3   Male 38     36-50  AU      Premium 199      0.04   Medium Tru
166. 3      4   Female 22     18-25  CA      Student 36      0.31   Low   Fals
167. 4      5   Other 29     26-35  US      Family 250      0.36   High   True

```

Selanjutnya, dibuat atribut turunan untuk memperkaya analisis. Kolom *age_group* digunakan untuk mengelompokkan pengguna berdasarkan rentang usia, sedangkan *engagement_level* dibuat dari *listening_time* untuk mengklasifikasikan tingkat aktivitas pengguna menjadi Low, Medium, dan High. Hasilnya adalah *dataset users* yang lebih terstruktur dengan 8000 baris dan 14 kolom, serta siap digunakan dalam analisis lanjutan dan pembentukan tabel dimensi pada *data warehouse*.

4.1.2.4. Overlaps

Bagian Cek Overlap *track_id* antara *Tracks & Playlist* ini bertujuan untuk memastikan keterkaitan data antara kedua *dataset*, yaitu apakah lagu yang ada di *tracks* juga muncul di *playlist*. Proses ini dilakukan dengan membandingkan nilai *track_id* dari kedua *dataset* menggunakan operasi himpunan.

Kode Program 8. Memperbaiki *overlap*

```

168. tracks_ids = set(df_tracks_clean["track_id"])
169. playlist_ids = set(df_playlist_clean["track_id"])
170. overlap = tracks_ids & playlist_ids
171. print(f"Overlap track_id (tracks ∩ playlist) : {len(overlap)}")
172. print(f"Track ID hanya di tracks           : {len(tracks_ids - playlist_ids)}")
173. print(f"Track ID hanya di playlist        : {len(playlist_ids - tracks_ids)}")

```

Output Program 6. Hasil cek overlap

```

174. Overlap track_id (tracks ∩ playlist) : 65
175. Track ID hanya di tracks           : 6122
176. Track ID hanya di playlist        : 1372

```

Hasilnya menunjukkan bahwa terdapat 65 *track_id* yang sama pada kedua *dataset*, sementara sebagian besar data hanya muncul di salah satu *dataset*, yaitu 6122 hanya di *tracks* dan 1372 hanya di *playlist*. Hal ini menunjukkan bahwa tidak semua lagu dalam *dataset tracks* masuk ke dalam *playlist*, dan sebaliknya. Tahap ini penting untuk memahami relasi antar data sebelum proses penggabungan (join) atau pembentukan tabel fakta pada *data warehouse*.

4.1.2.5. Tabel *Dimension*

Tabel dimensi (*dimension table*) adalah tabel di dalam *data warehouse* yang berisi informasi kontekstual untuk menjelaskan data angka yang sedang dianalisis. Proses pembentukan tabel dimensi (*dimension table*) dilakukan untuk mengekstrak, mengintegrasikan, dan menyusun data kontekstual dari berbagai *dataset* yang telah dibersihkan agar siap digunakan dalam arsitektur *data warehouse*. Pada tahap ini, data akan ditransformasikan ke dalam empat struktur dimensi utama yang saling melengkapi

a. Dimensi *Track*

Bagian *track* ini bertujuan untuk membentuk tabel dimensi *track* yang akan digunakan dalam *data warehouse*. Proses dimulai dengan mengambil data utama dari *dataset playlist*, kemudian diperkaya dengan informasi tambahan dari *dataset tracks* seperti genre dan explicit melalui proses join berdasarkan *track_id*.

Kode Program 9. Buat *dim_track*

```

177. dim_track_base = df_playlist_clean[[
178.     "track_id", "track_name", "artist_name", "album_name",
179.     "energy", "tempo", "danceability", "loudness", "liveness",
180.     "valence", "acousticness", "instrumentalness", "speechiness",
181.     "key", "mode", "duration_ms", "duration_min", "track_popularity"
182. ]].drop_duplicates(subset="track_id").copy()
183.

```

```

184.     tracks_enrich = df_tracks_clean[["track_id", "genre", "explicit"]].copy()
185.     dim_track = dim_track_base.merge(tracks_enrich, on="track_id", how="left")
186.
187.     dim_track["explicit"] = dim_track["explicit"].fillna(False)
188.     dim_track["genre"] = dim_track["genre"].fillna("Unknown")
189.     dim_track["popularity_level"] = pd.cut(
190.         dim_track["track_popularity"],
191.         bins=[-1, 30, 70, 100],
192.         labels=["Low", "Medium", "High"]
193.     ).astype(str)
194.
195.     tracks_only = df_tracks_clean[~df_tracks_clean["track_id"].isin(dim_track["track_id"])].copy()
196.     tracks_only["genre"] = tracks_only.get("genre", "Unknown")
197.     tracks_only["explicit"] = tracks_only["explicit"].fillna(False)
198.     for col in ["energy", "tempo", "danceability", "loudness", "liveness", "valence", "acousticness", "instrumentalness", "speechiness", "key", "mode"]:
199.         tracks_only[col] = np.nan
200.
201.     tracks_only = tracks_only[[
202.         "track_id", "track_name", "artist_name", "album_name",
203.         "energy", "tempo", "danceability", "loudness", "liveness",
204.         "valence", "acousticness", "instrumentalness", "speechiness",
205.         "key", "mode", "duration_ms", "duration_min",
206.         "track_popularity", "genre", "explicit", "popularity_level"
207.     ]]
208.
209.     dim_track = pd.concat([dim_track, tracks_only],
210.         ignore_index=True).drop_duplicates(subset="track_id")
211.     print("Dim_Track:", dim_track.shape)
212.     display(dim_track.head())

```

Selanjutnya dilakukan pembersihan data, seperti mengisi nilai yang kosong pada kolom genre dan explicit, serta memastikan konsistensi tipe data. Atribut turunan seperti popularity_level juga ditambahkan untuk mengelompokkan tingkat popularitas lagu. Data dari *tracks* yang tidak terdapat di *playlist* juga ditambahkan agar tabel dimensi menjadi lebih lengkap. Duplikasi berdasarkan *track_id* kemudian dihapus sehingga setiap lagu hanya muncul satu kali.

b. Dimensi *Playlist*

Bagian *playlist* ini bertujuan untuk membentuk tabel dimensi *playlist* yang berisi informasi unik terkait *playlist*. Data diambil dari *dataset playlist* yang sudah dibersihkan, kemudian hanya kolom yang relevan seperti *playlist_id*, *playlist_name*, *playlist_genre*, dan *playlist_subgenre* yang dipilih.

Kode Program 10. Buat *dim_playlist*

```
212.     dim_playlist = df_playlist_clean[[
213.         "playlist_id", "playlist_name", "playlist_genre", "playlist_subgenre"
214.     ]].drop_duplicates(subset="playlist_id").reset_index(drop=True)
215.
216.     print("Dim_Playlist:", dim_playlist.shape)
217.     display(dim_playlist.head())
```

Output Program 7. Pratinjau *dim_playlist*

```
218.     Dim_Playlist: (72, 4)
219.     playlist_id    playlist_name    playlist_genre    playlist_subgenre
220.     0              37i9dQZF1DXcBWIGoYBM5M Today's Top Hits      pop      mainstream
221.     1              37i9dQZF1DWXRqgorJj26U Rock Classics      rock      classic
222.     2              37i9dQZF1DXbITWG1ZJKYt Jazz Classics      jazz      classic
223.     3              37i9dQZF1DWWEJlAGA9gs0 Classical Essentials  classical    essential
224.     4              37i9dQZF1DX0XUsuxWHRQd RapCaviar      hip-hop modern
```

Selanjutnya dilakukan penghapusan data duplikat berdasarkan *playlist_id* agar setiap *playlist* hanya muncul satu kali.

c. Dimensi User

Bagian *user* ini bertujuan untuk membentuk tabel dimensi *user* yang berisi informasi karakteristik pengguna. Data diambil dari *dataset users* yang sudah melalui proses pembersihan dan transformasi sebelumnya, kemudian dipilih kolom-kolom penting seperti *user_id*, *gender*, *age_group*, *country*, *subscription_type*, hingga *engagement_level*.

Kode Program 11. Buat *dim_user*

```
225.     dim_user = df_users_clean[[
226.         "user_id", "gender", "age", "age_group", "country",
227.         "subscription_type", "device_type", "offline_listening",
228.         "ads_listened_per_week", "is_churned", "engagement_level"
229.     ]].drop_duplicates(subset="user_id").reset_index(drop=True)
230.
231.     print("Dim_User:", dim_user.shape)
232.     display(dim_user.head())
```

Output Program 8. Hasil *dim_user*

233.	Dim_User: (8000, 11)									
234.	user_id	gender	age	age_group	country	subscription_type	device_type			
	offline_listening			ads_listened_per_week	is_churned	engagement_level				
235.	0	1	Female	54	50+	CA	Free	Desktop	False	31
	True	Low								
236.	1	2	Other	33	26-35	DE	Family	Web	True	0
	False	Medium								
237.	2	3	Male	38	36-50	AU	Premium	Mobile	True	0
	True	Medium								
238.	3	4	Female	22	18-25	CA	Student	Mobile	True	0
	False	Low								
239.	4	5	Other	29	26-35	US	Family	Mobile	True	0
	True	High								

Tabel dimensi terbentuk dengan total 8.000 baris data unik dan 11 kolom atribut. Proses penghapusan duplikasi menggunakan fungsi `drop_duplicates` berbasis `user_id` memastikan bahwa setiap pengguna hanya tercatat satu kali.

d. Dimensi *Time*

Bagian *time* ini bertujuan untuk membentuk tabel dimensi waktu yang akan digunakan dalam analisis berbasis waktu pada *data warehouse*. Data tanggal diambil dari kolom `track_album_release_date` pada *dataset playlist*, kemudian ditentukan rentang tanggal dari yang paling awal hingga paling akhir.

Kode Program 12. Pembuatan *dim_time*

```

240.         release_dates = df_playlist_clean["track_album_release_date"].dropna()
241.
242.         date_min = release_dates.min()
243.         date_max = pd.Timestamp("2025-12-31")
244.         date_range = pd.date_range(start=date_min, end=date_max, freq="D")
245.
246.         dim_time = pd.DataFrame({
247.             "time_id": range(1, len(date_range) + 1),
248.             "date": date_range,
249.             "day": date_range.day,
250.             "month": date_range.month,
251.             "year": date_range.year,
252.             "quarter": date_range.quarter,
253.             "weekday": date_range.day_name()
254.         })
255.
256.         print("Dim_Time:", dim_time.shape)
257.         display(dim_time.head())
258.         print(f"Range         tanggal:         {dim_time['date'].min().date()}         s.d.
           {dim_time['date'].max().date()}")

```

Output Program 9. Hasil pembuatan dimensi waktu

```

259.         Dim_Time: (26298, 7)
260.         time_id date    day    month    year    quarter weekday
261.         0         1      1954-01-01    1         1      1954    1      Friday
262.         1         2      1954-01-02    2         1      1954    1      Saturday
263.         2         3      1954-01-03    3         1      1954    1      Sunday
264.         3         4      1954-01-04    4         1      1954    1      Monday
265.         4         5      1954-01-05    5         1      1954    1      Tuesday
266.         Range tanggal: 1954-01-01 s.d. 2025-12-31

```

Selanjutnya dibuat tabel waktu yang berisi urutan tanggal secara lengkap, lalu diturunkan menjadi beberapa atribut seperti day, month, year, quarter, dan weekday. Setiap tanggal juga diberikan time_id sebagai identifier unik. Hasil akhirnya adalah tabel dimensi waktu yang terstruktur dan lengkap, sehingga dapat digunakan untuk analisis tren berdasarkan waktu.

4.1.2.6. Tabel *Facts*

Setelah seluruh tabel dimensi berhasil dibentuk dan dibersihkan, tahap selanjutnya adalah menyusun *fact table* sebagai pusat dari arsitektur *star schema* dalam *data warehouse*. Tabel fakta ini dirancang untuk merekam data transaksional atau metrik kuantitatif dari aktivitas *streaming* musik, seperti akumulasi durasi pemutaran dan jumlah lagu yang didengarkan. Struktur tabel ini secara khusus mengintegrasikan kumpulan *foreign keys* yang merujuk langsung pada primary key dari dim_track, dim_playlist, dim_user, dan dim_time.

Tabel `fact_listening` ini bertujuan untuk membentuk tabel fakta yang merepresentasikan aktivitas pengguna dalam mendengarkan musik. Data dibuat dengan menggabungkan informasi dari beberapa sumber, seperti *user*, *track*, *playlist*, dan waktu, sehingga menghasilkan data yang saling terhubung.

Kode Program 13. Pembuatan tabel fakta

```

267.         np.random.seed(42)
268.         N_FACTS = 50000
269.
270.         track_lookup = dim_track.set_index("track_id")["track_popularity",
"duration_ms"].copy()
271.         track_lookup["track_popularity"] =
track_lookup["track_popularity"].fillna(50).astype(int)
272.         track_lookup["duration_ms"] =
track_lookup["duration_ms"].fillna(200000).astype(int)
273.
274.         user_stats_lookup = df_users_clean.set_index("user_id")[[
275.             "listening_time", "songs_played_per_day", "skip_rate",
"ads_listened_per_week"
276.         ]].copy()
277.
278.         user_weights = df_users_clean.set_index("user_id")["listening_time"]
279.         user_weights = user_weights / user_weights.sum()
280.
281.         sampled_user_ids = np.random.choice(user_weights.index.values, size=N_FACTS,
p=user_weights.values)
282.         sampled_playlist_ids = np.random.choice(dim_playlist["playlist_id"].values,
size=N_FACTS)
283.
284.         playlist_track_flat = df_playlist_clean[["playlist_id",
"track_id"]].drop_duplicates()
285.         pool_playlist_tracks = playlist_track_flat["track_id"].values
286.         pool_all_tracks = dim_track["track_id"].values
287.
288.         use_playlist_pool = np.random.rand(N_FACTS) < 0.70
289.         sampled_track_ids = np.where(
290.             use_playlist_pool,
291.             np.random.choice(pool_playlist_tracks, size=N_FACTS),
292.             np.random.choice(pool_all_tracks, size=N_FACTS)
293.         )
294.         time_pool = dim_time[dim_time["year"].between(2020, 2025)]["time_id"].values
295.         sampled_time_ids = np.random.choice(time_pool, size=N_FACTS)

```

Proses pembentukan tabel fakta diawali dengan melakukan persiapan data referensi dari tabel dimensi yang telah dibentuk sebelumnya, serta melakukan pembobotan distribusi data acak untuk mensimulasikan riwayat transaksi pendengar secara realistis. Setelah seluruh data sampel ID dan bobot statistik berhasil disiapkan, langkah selanjutnya adalah menyusun

struktur fisik DataFrame tabel fakta, melakukan penggabungan (*merge*) dengan data metrik, serta mengalkulasi atribut metrik baru.

Kode Program 14. Penggabungan atribut

```
296.     fact_listening = pd.DataFrame({
297.         "listening_id": range(1, N_FACTS + 1),
298.         "user_id": sampled_user_ids,
299.         "track_id": sampled_track_ids,
300.         "playlist_id": sampled_playlist_ids,
301.         "time_id": sampled_time_ids
302.     })
303.
304.     fact_listening = fact_listening.merge(
305.         user_stats_lookup, left_on="user_id", right_index=True, how="left"
306.     )
307.     fact_listening = fact_listening.merge(
308.         track_lookup, left_on="track_id", right_index=True, how="left"
309.     )
310.
311.     fact_listening["completed_rate"] = np.clip(
312.         fact_listening["listening_time"] / (fact_listening["duration_ms"] / 1000),
313.         0, 1
314.     ).round(4)
315.
316.     fact_listening = fact_listening[[
317.         "listening_id", "user_id", "track_id", "playlist_id", "time_id",
318.         "listening_time", "songs_played_per_day", "skip_rate",
319.         "ads_listened_per_week", "track_popularity", "completed_rate"
320.     ]]
321.
322.     print("Fact_Listening:", fact_listening.shape)
323.     display(fact_listening.head())
```

Pengambilan sampel data pengguna, lagu, dan *playlist*, kemudian digabungkan dengan atribut tambahan seperti *listening_time*, *songs_played_per_day*, *skip_rate*, dan *ads_listened_per_week*. Untuk tambahan juga atribut turunan seperti *completed_rate* untuk merepresentasikan rasio durasi lagu yang diselesaikan oleh pengguna. Tabel fakta *fact_listening* dibentuk dengan total 50.000 baris data transaksi transaksi dan 11 kolom atribut. Struktur tabel fakta ini secara presisi mengintegrasikan seluruh dimensi yang ada melalui kolom foreign keys berupa *user_id*, *track_id*, *playlist_id*, dan *time_id*.

Output Program 10. *Preview hasil generate tabel fact*

324.	Fact_Listening: (50000, 11)							
325.	listening_id	user_id	track_id	playlist_id	time_id	listening_time		
	songs_played_per_day		skip_rate	ads_listened_per_week		track_popularity		
	completed_rate							
326.	0	1	2980	0JPzG00AY7hNVu8sauadAu	37i9dQZF1DX10zKzsJ2jva	25847	234	
	69	0.26	44	28	0.9289			
327.	1	2	7600	1x4eRmHQgpYMurqlvopfd3	37i9dQZF1E1cLyY4fNNQaC	25011	148	
	90	0.10	0	24	0.9873			
328.	2	3	5840	7J2gyNghNTz14EsLhXp01Q	37i9dQZF1DX186v583rmzp	26091	262	
	17	0.18	27	70	1.0000			
329.	3	4	4767	4q05KbxYZ5JdkmrletgPjF	37i9dQZF1E1dOtkojz5FWw	24608	99	
	12	0.51	0	69	0.5441			
330.	4	5	1212	0045fw2L5vsWpdsOdXwNAR	37i9dQZF1DXaXdsfv6nvZ5	25787	51	
	30	0.03	0	80	0.2101			

4.1.3. Load Data

Proses Load merupakan tahap akhir dari siklus ETL yang berfungsi untuk memindahkan dan menyimpan data yang telah dibersihkan secara permanen ke dalam sistem penyimpanan target. Bagian *Load* ke SQLite ini bertujuan untuk menyimpan seluruh hasil transformasi ke dalam *database* SQLite sebagai *data warehouse*. Pada tahap ini, terlebih dahulu dibuat struktur tabel yang terdiri dari tabel dimensi (*dim_time*, *dim_user*, *dim_playlist*, *dim_track*) dan tabel fakta beserta relasi antar tabel melalui *foreign key*.

Kode Program 15. Pembuatan struktur *database*

```

331.         conn = sqlite3.connect(DB_PATH)
332.         cursor = conn.cursor()
333.
334.         tables = ["Fact_Listening", "Dim_Track", "Dim_Playlist", "Dim_User", "Dim_Time"]
335.         for t in tables:
336.             cursor.execute(f"DROP TABLE IF EXISTS {t}")
337.         conn.commit()
338.
339.         cursor.executescript("""
340.         CREATE TABLE Dim_Time (
341.             time_id    INTEGER PRIMARY KEY,
342.             date        TEXT,
343.             day          INTEGER,
344.             month        INTEGER,
345.             year          INTEGER,
346.             quarter      INTEGER,
347.             weekday      TEXT
348.         );

```

```

349.      CREATE TABLE Dim_User (
350.          user_id          INTEGER PRIMARY KEY,
351.          gender            TEXT,
352.          age               INTEGER,
353.          age_group         TEXT,
354.          country           TEXT,
355.          subscription_type TEXT,
356.          device_type       TEXT,
357.          offline_listening INTEGER,
358.          ads_listened_per_week INTEGER,
359.          is_churned        INTEGER,
360.          engagement_level  TEXT
361.      );
362.
363.      CREATE TABLE Dim_Playlist (
364.          playlist_id      TEXT PRIMARY KEY,
365.          playlist_name    TEXT,
366.          playlist_genre   TEXT,
367.          playlist_subgenre TEXT
368.      );
369.
370.      CREATE TABLE Dim_Track (
371.          track_id          TEXT PRIMARY KEY,
372.          track_name        TEXT,
373.          artist_name       TEXT,
374.          album_name        TEXT,
375.          energy            REAL,
376.          tempo             REAL,
377.          danceability      REAL,
378.          loudness          REAL,
379.          liveness          REAL,
380.          valence           REAL,
381.          acousticness      REAL,
382.          instrumentalness  REAL,
383.          speechiness       REAL,
384.          key               REAL,
385.          mode              REAL,
386.          duration_ms       REAL,
387.          duration_min      REAL,
388.          track_popularity  INTEGER,
389.          genre             TEXT,
390.          explicit          INTEGER,
391.          popularity_level  TEXT
392.      );
393.
394.      CREATE TABLE Fact_Listening (
395.          listening_id      INTEGER PRIMARY KEY,
396.          user_id           INTEGER,
397.          track_id          TEXT,
398.          playlist_id       TEXT,

```

```

399.         time_id            INTEGER,
400.         listening_time      REAL,
401.         songs_played_per_day REAL,
402.         skip_rate           REAL,
403.         ads_listened_per_week REAL,
404.         track_popularity     INTEGER,
405.         completed_rate       REAL,
406.         FOREIGN KEY (user_id) REFERENCES Dim_User(user_id),
407.         FOREIGN KEY (track_id) REFERENCES Dim_Track(track_id),
408.         FOREIGN KEY (playlist_id) REFERENCES Dim_Playlist(playlist_id),
409.         FOREIGN KEY (time_id) REFERENCES Dim_Time(time_id)
410.     );
411.     """
412.
413.     dim_time.to_sql("Dim_Time", conn, if_exists="append", index=False)
414.     dim_user.to_sql("Dim_User", conn, if_exists="append", index=False)
415.     dim_playlist.to_sql("Dim_Playlist", conn, if_exists="append", index=False)
416.     dim_track.to_sql("Dim_Track", conn, if_exists="append", index=False)
417.     fact_listening.to_sql("Fact_Listening", conn, if_exists="append", index=False)
418.     conn.commit()
419.
420.     for t in tables:
421.         count = pd.read_sql(f"SELECT COUNT(*) AS total_rows FROM {t}", conn)
422.         print(f"{t}:")
423.         display(count)
424.
425.     conn.close()
426.     print(f>Data warehouse berhasil disimpan ke: {DB_PATH}")

```

Output Program 11. Preview hasil generate tabel fact

```

427.     Fact_Listening:
428.     total_rows
429.     0          50000
430.     Dim_Track:
431.     total_rows
432.     0          7559
433.     Dim_Playlist:
434.     total_rows
435.     0          72
436.     Dim_User:
437.     total_rows
438.     0          8000
439.     Dim_Time:
440.     total_rows
441.     0          26298
442.     Data warehouse berhasil disimpan ke: /content/drive/MyDrive/dataset
443.     BIawak/spotify_dw.db

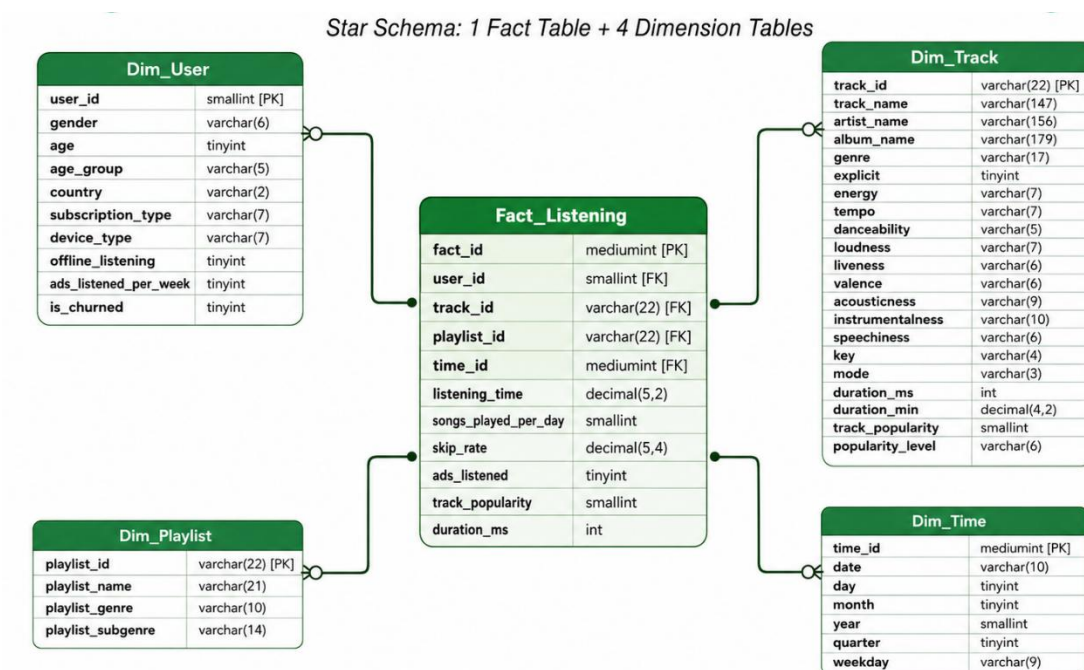
```

Langkah awal dilakukan dengan menghapus tabel lama jika ada (*drop table*), dilanjutkan dengan mendefinisikan struktur skema *Star Schema*, lalu memuat seluruh data ke dalam *database* SQLite. Berdasarkan *log* akhir program, seluruh tabel dimensi dan tabel fakta berhasil dibuat dan diisi dengan data secara lengkap. Verifikasi jumlah baris data menggunakan perintah `COUNT(*)` memastikan bahwa proses transfer data ke dalam file *database* target di `DB_PATH` telah berjalan sukses tanpa ada kehilangan data.

Struktur tabel dibuat dengan data dari masing-masing *DataFrame* kemudian dimasukkan ke dalam tabel yang sesuai menggunakan fungsi `to_sql()`. Proses ini memastikan seluruh data yang telah dibersihkan dan diolah tersimpan secara terstruktur dalam *database*. Validasi dilakukan pengecekan jumlah baris pada setiap tabel untuk memastikan data berhasil dimuat dengan benar. Hasilnya menunjukkan bahwa seluruh tabel telah terisi sesuai dengan jumlah data yang diharapkan. Tahap ini menandai bahwa proses ETL telah selesai dan *data warehouse* siap digunakan untuk analisis lebih lanjut.

4.2 Hasil Akhir *Data warehouse*

Hasil akhir dari proses perancangan dan pembangunan *data warehouse* pada proyek ini adalah terbentuknya sebuah skema *data warehouse* yang dapat digunakan untuk menganalisis perilaku pengguna dalam mendengarkan musik pada platform Spotify.



Gambar 4. 1 Struktur *database*

Data warehouse ini disusun menggunakan pendekatan *star schema*, yaitu model data yang terdiri atas satu tabel fakta sebagai pusat data transaksi dan beberapa tabel dimensi sebagai pendukung analisis.

Table	Action	Rows	Type	Collation	Size	Overhead
☐ dim_playlist	★ Browse Structure Search Insert Empty Drop	72	InnoDB	utf8mb4_0900_ai_ci	16.0 KiB	-
☐ dim_time	★ Browse Structure Search Insert Empty Drop	26,298	InnoDB	utf8mb4_0900_ai_ci	1.5 MiB	-
☐ dim_track	★ Browse Structure Search Insert Empty Drop	7,559	InnoDB	utf8mb4_0900_ai_ci	1.5 MiB	-
☐ dim_user	★ Browse Structure Search Insert Empty Drop	8,000	InnoDB	utf8mb4_0900_ai_ci	16.0 KiB	-
☐ fact_listening	★ Browse Structure Search Insert Empty Drop	50,000	InnoDB	utf8mb4_0900_ai_ci	16.0 KiB	-
5 tables	Sum	91,929	InnoDB	utf8mb4_0900_ai_ci	3.1 MiB	0 B

Gambar 4. 2 Tabel di *database*

Data warehouse yang dihasilkan terdiri dari satu tabel fakta, yaitu Fact_Listening, serta empat tabel dimensi, yaitu Dim_User, Dim_Track, Dim_Playlist, dan Dim_Time. Tabel fakta menyimpan data aktivitas mendengarkan musik, sedangkan tabel dimensi menyimpan informasi deskriptif yang digunakan untuk melihat data dari berbagai sudut pandang, seperti pengguna, lagu, *playlist*, dan waktu.

Table 1. Hasil *generate database*

Nama Tabel	Jenis Tabel	Jumlah Data	Keterangan
Dim_User	Dimensi	8.000 data	Menyimpan informasi pengguna, seperti gender, usia, negara, jenis langganan, perangkat, dan status <i>churn</i>
Dim_Track	Dimensi	4.027 data	Menyimpan informasi lagu, seperti nama lagu, artis, album, genre, durasi, popularitas, serta atribut audio
Dim_Playlist	Dimensi	72 data	Menyimpan informasi <i>playlist</i> , seperti nama <i>playlist</i> , genre <i>playlist</i> , dan subgenre <i>playlist</i>
Dim_Time	Dimensi	26.298 data	Menyimpan informasi waktu, seperti tanggal, hari, bulan, tahun, kuartal, dan nama hari
Fact_Listening	Fakta	50.000 data	Menyimpan aktivitas mendengarkan musik yang menghubungkan <i>user</i> , <i>track</i> , <i>playlist</i> , dan waktu

BAB V

HASIL ANALISIS

5.1 Analisis Data

Tahap analisis data menggunakan pendekatan *Online Analytical Processing* (OLAP) dilakukan untuk mengekstrak informasi dan wawasan bisnis dari data historis yang telah tersimpan di dalam *data warehouse*. Guna memfasilitasi proses penarikan data tersebut secara efisien, sebuah fungsi utilitas bernama `run_query()` dikembangkan untuk menjembatani sistem. Fungsi ini mengotomatisasi eksekusi perintah SQL kueri multi-dimensi langsung ke basis data SQLite, kemudian mentransformasikan kumpulan baris hasil.

Kode Program 16. Fungsi *run query*

```
442.     def run_query(sql):
443.         conn = sqlite3.connect(DB_PATH)
444.         result = pd.read_sql(sql, conn)
445.         conn.close()
446.         return result
```

Fungsi ini mempermudah proses analisis karena pengguna hanya perlu menuliskan *query* SQL tanpa harus berulang kali membuka dan menutup koneksi *database*.

5.1.1. Rata-Rata Popularitas Berdasarkan Genre

Analisis ini dilakukan untuk mengetahui jenis genre musik yang paling diminati oleh pengguna secara global berdasarkan metrik popularitas lagu dengan cara mengagregasikan nilai rata-rata popularitas pada setiap genre, sehingga dapat memetakan tren pasar saat ini guna mengoptimalkan strategi kurasi lagu dan rekomendasi musik.

Kode Program 17. *Top genres*


```

447.      q61 = '''
448.      SELECT
449.          t.genre,
450.          COUNT(*)                AS total_plays,
451.          ROUND(AVG(f.track_popularity),2) AS avg_popularity,
452.          ROUND(AVG(f.skip_rate),4)      AS avg_skip_rate,
453.          ROUND(AVG(f.listening_time),2) AS avg_listening_time

```

```

454.      FROM Fact_Listening f
455.      JOIN Dim_Track t ON f.track_id = t.track_id
456.      WHERE t.genre != 'Unknown'
457.      GROUP BY t.genre
458.      ORDER BY avg_popularity DESC
459.      LIMIT 10
460.      '''
461.      result61 = run_query(q61)
462.      display(result61)

```

Genre rock menduduki peringkat tertinggi dengan tingkat popularitas rata-rata (avg_popularity) sebesar 70,20 dan jumlah pemutaran (total_plays) sebanyak 243 kali. Menariknya, meskipun genre pop memiliki popularitas yang cukup tinggi (62,85), genre ini mencatat rasio lewatan tertinggi (avg_skip_rate) sebesar 0,3329, yang mengindikasikan kecenderungan pengguna untuk beralih lagu lebih cepat.

Output Program 12. Top genres

	genre	total_plays	avg_popularity	avg_skip_rate	avg_listening_time
463.	0 rock	243	70.20	0.3020	207.98
464.	1 summer	234	69.29	0.3031	201.29
465.	2 dance	185	66.74	0.3130	191.91
466.	3 pop	116	62.85	0.3329	200.03
467.	4 rock-n-roll	149	61.96	0.2866	193.42
468.	5 sleep	138	60.69	0.3128	197.51
469.	6 country	113	59.28	0.2807	203.10
470.	7 r-n-b	224	58.40	0.3002	199.86
471.	8 party	115	58.03	0.3092	200.44
472.	9 reggaeton	164	57.76	0.3137	204.20

Genre country mencatat loyalitas pendengar yang paling stabil dengan rasio lewatan terendah (0,2807). Dilihat dari segi durasi konsumsi, genre rock kembali unggul dengan rata-rata waktu dengar (avg_listening_time) mencapai 207,98 detik, disusul erat oleh genre reggaeton sebesar 204,20 detik.

5.1.2. Tingkat *Churn* Berdasarkan Tipe Langganan

Analisis ini dilakukan untuk mengetahui tingkat *churn* pengguna berdasarkan jenis langganan yang digunakan dengan membandingkan nilai rata-rata *skip rate* dan *listening time* untuk melihat apakah tingkat *churn* berkaitan dengan rendahnya keterlibatan pengguna dalam mendengarkan musik.

Kode Program 18. *Churn Rate by Subscription Type*

```
474. q62 = ''
475. SELECT
476.     u.subscription_type,
477.     COUNT(DISTINCT u.user_id)      AS total_users,
478.     SUM(u.is_churned)              AS churned_users,
479.     ROUND(AVG(u.is_churned)*100, 2) AS churn_rate_pct,
480.     ROUND(AVG(f.skip_rate), 4)     AS avg_skip_rate,
481.     ROUND(AVG(f.listening_time), 2) AS avg_listening_time
482. FROM Dim_User u
483. JOIN Fact_Listening f ON u.user_id = f.user_id
484. GROUP BY u.subscription_type
485. ORDER BY churn_rate_pct DESC
486. ''
487. result62 = run_query(q62)
488. display(result62)
```

Langganan Family memiliki tingkat *churn* tertinggi dengan nilai *churn_rate_pct* sebesar 26,79%. Jumlah pengguna pada kategori ini sebanyak 1.795 pengguna, dengan jumlah *churned users* sebesar 3.135, dan memiliki nilai *avg_skip_rate* tertinggi, yaitu 0,3028, serta rata-rata waktu dengar paling rendah, yaitu 197,87 detik.

Output Program 13. *Churn rate*

489.	<i>subscription_type</i>	<i>total_users</i>	<i>churned_users</i>	<i>churn_rate_pct</i>	<i>avg_skip_rate</i>
	<i>avg_listening_time</i>				
490.	-----	-----:	-----:	-----:	-----:
	-----:				
491.	Family	1795	3135	26.79	0.3028
	197.87				
492.	Student	1828	3274	26.60	0.3020
	201.07				
493.	Premium	1992	3374	25.33	0.2909
	201.21				
494.	Free	1917	3109	24.54	0.3001
	200.41				

Free memiliki tingkat *churn* terendah, yaitu 24,54%, meskipun nilai *skip rate*-nya masih cukup tinggi sebesar 0,3001. Jenis langganan Premium menunjukkan performa *engagement* yang paling baik karena memiliki *avg_skip_rate* terendah sebesar 0,2909 dan *avg_listening_time* tertinggi sebesar 201,21 detik.

5.1.3. Perilaku Mendengarkan di 10 Negara Teratas

Di sini dilakukan analisa untuk mengetahui negara dengan aktivitas mendengarkan musik paling tinggi berdasarkan jumlah pemutaran lagu. Sekaligus mempertimbangkan rata-rata waktu dengar, *skip rate*, rata-rata lagu yang diputar per hari, serta *churn rate*.

Kode Program 19. Churn Rate by Subscription Type

```

495. q63 = ''
496. SELECT
497.     u.country,
498.     COUNT(*)                               AS total_plays,
499.     ROUND(AVG(f.listening_time), 2)        AS avg_listening_time,
500.     ROUND(AVG(f.skip_rate), 4)             AS avg_skip_rate,
501.     ROUND(AVG(f.songs_played_per_day),2)   AS avg_songs_per_day,
502.     ROUND(AVG(u.is_churned)*100, 2)        AS churn_rate_pct
503. FROM Fact_Listening f
504. JOIN Dim_User u ON f.user_id = u.user_id
505. GROUP BY u.country
506. ORDER BY total_plays DESC
507. LIMIT 10
508. ''
509. result63 = run_query(q63)
510. display(result63)

```

United States (US/Amerika Serikat) memiliki jumlah pemutaran tertinggi dengan total 6.507 *plays*. Hal ini menunjukkan bahwa pengguna dari Amerika Serikat merupakan kelompok pengguna yang paling aktif berdasarkan jumlah aktivitas pemutaran lagu. Jika

dilihat dari rata-rata waktu dengar, negara Germany (DE/Jerman) memiliki nilai tertinggi, yaitu 203,48 detik, meskipun total pemutarannya berada pada peringkat kedua dengan 6.401 *plays*.

Output Program 14. Listening Behavior

511.	country	total_plays	avg_listening_time	avg_skip_rate	avg_songs_per_day	churn_rate_pct
512.	-----	-----:	-----:	-----:	-----:	---
	-----:					
513.	US	6507	199.19	0.2945	48.86	
	27.06					
514.	DE	6401	203.48	0.2998	47.46	
	26.15					
515.	AU	6346	198.13	0.3000	49.56	
	25.76					
516.	PK	6337	199.95	0.3030	51.36	
	27.54					

Pakistan (PK) memiliki nilai *churn_rate_pct* tertinggi sebesar 27,54% serta *avg_skip_rate* tertinggi sebesar 0,3030. Hal ini mengindikasikan adanya potensi penurunan *engagement* pada pengguna dari Pakistan, karena pengguna cenderung lebih sering melewati lagu dan memiliki risiko *churn* yang lebih tinggi dibandingkan negara lain.

5.1.4. Hubungan Fitur Audio dengan *Skip Rate*

Skip rate adalah kecenderungan pengguna dalam melewati lagu. Karakteristik audio yang digunakan dalam analisis ini meliputi energy, danceability, tempo, dan valence. Data kemudian dikelompokkan berdasarkan *Popularity Level* untuk melihat apakah lagu dengan tingkat popularitas tertentu memiliki *skip rate* yang lebih rendah atau lebih tinggi.

Kode Program 20. Audio dan *skip rate*

```

517. q64 = '''
518. SELECT
519.     t.popularity_level,
520.     ROUND(AVG(t.energy),3)      AS avg_energy,
521.     ROUND(AVG(t.danceability),3) AS avg_danceability,
522.     ROUND(AVG(t.tempo),2)      AS avg_tempo,
523.     ROUND(AVG(t.valence),3)    AS avg_valence,
524.     ROUND(AVG(f.skip_rate),4)  AS avg_skip_rate,
525.     COUNT(*)                   AS total_plays
526. FROM Fact_Listening f
527. JOIN Dim_Track t ON f.track_id = t.track_id
528. WHERE t.popularity_level NOT IN ('None', 'nan')
529. GROUP BY t.popularity_level
530. ORDER BY avg_skip_rate ASC
531. '''
532. result64 = run_query(q64)
533. display(result64)

```

Medium memiliki nilai `avg_skip_rate` terendah sebesar 0,2979 dengan total pemutaran sebanyak 13.590 *plays*. Lagu pada kategori ini memiliki rata-rata `energy` sebesar 0,647, `danceability` sebesar 0,663, `tempo` sebesar 120,87, dan `valence` sebesar 0,533. Nilai *skip rate* yang paling rendah menunjukkan bahwa lagu dengan tingkat popularitas menengah cenderung lebih jarang dilewati oleh pengguna. Kategori High memiliki total pemutaran tertinggi, yaitu 29.787 *plays*, tetapi justru memiliki nilai *skip rate* paling tinggi sebesar 0,2993. Hal ini menunjukkan bahwa popularitas lagu yang tinggi tidak selalu menjamin bahwa lagu tersebut memiliki tingkat retensi pendengar yang lebih baik.

Output Program 15. Audio features vs Skip rate

```

534. | popularity_level | avg_energy | avg_danceability | avg_tempo | avg_valence |
535. | -----: | -----: | -----: | -----: | -----: |
536. | Medium          | 0.647 | 0.663 | 120.87 | 0.533 |
537. | Low             | NaN | NaN | NaN | NaN |
538. | High           | 0.671 | 0.650 | 121.06 | 0.526 |

```

5.1.5. Tren Jumlah Pemutaran Lagu per Tahun

Di sini kita akan mengetahui perkembangan jumlah pemutaran lagu dari tahun ke tahun. Penggunaan dimensi waktu dapat digunakan untuk melihat tren aktivitas pengguna

secara periodik. Analisa menampilkan rata-rata waktu dengar, rata-rata popularitas lagu, dan *skip rate* pada setiap tahun.

Kode Program 21. *Trend Plays per Year*

```

539.      q65 = '''
540.      SELECT
541.          tm.year,
542.          COUNT(*)                AS total_plays,
543.          ROUND(AVG(f.listening_time), 2) AS avg_listening_time,
544.          ROUND(AVG(f.track_popularity), 2) AS avg_track_popularity,
545.          ROUND(AVG(f.skip_rate), 4)      AS avg_skip_rate
546.      FROM Fact_Listening f
547.      JOIN Dim_Time tm ON f.time_id = tm.time_id
548.      GROUP BY tm.year
549.      ORDER BY tm.year
550.      '''
551.      result65 = run_query(q65)
552.      display(result65)

```

Pemutaran lagu setiap tahun berada pada rentang yang relatif stabil, yaitu antara 8.206 hingga 8.495 *plays*. Meskipun terdapat perbedaan jumlah pemutaran, selisih antar tahun tidak terlalu besar sehingga pola aktivitas pengguna dapat dikatakan cukup stabil. Tahun 2022 memiliki rata-rata waktu dengar tertinggi sebesar 201,71 detik, sementara tahun 2023 memiliki rata-rata waktu dengar terendah sebesar 198,24 detik. *Skip rate* tahun 2024 memiliki nilai terendah sebesar 0,2972, sedangkan tahun 2021 memiliki nilai tertinggi sebesar 0,3010.

Output Program 16. *Trend Plays per Year*

553.	year	total_plays	avg_listening_time	avg_track_popularity	avg_skip_rate
554.	---:	-----:	-----:	-----:	-----:
555.	2020	8232	201.56	64.29	0.2990
556.	2021	8495	199.64	64.46	0.3010
557.	2022	8206	201.71	64.59	0.2973
558.	2023	8299	198.24	64.33	0.2988
559.	2024	8397	199.96	64.81	0.2972
560.	2025	8371	200.08	64.71	0.2993

5.1.6. Korelasi *Churn* Pengguna dengan Fitur Audio

Bagian ini membandingkan perilaku pengguna aktif dan pengguna *churned* berdasarkan karakteristik audio lagu yang mereka dengarkan, guna mengetahui apakah *churn* lebih dipengaruhi oleh jenis musik yang didengarkan atau oleh tingkat *engagement* pengguna.

Kode Program 22. Churn vs Audio Features

```
561. q66 = ''
562. SELECT
563.     CASE u.is_churned WHEN 1 THEN 'Churned' ELSE 'Active' END AS user_status,
564.     ROUND(AVG(t.energy),3) AS avg_energy,
565.     ROUND(AVG(t.danceability),3) AS avg_danceability,
566.     ROUND(AVG(t.valence),3) AS avg_valence,
567.     ROUND(AVG(t.tempo),2) AS avg_tempo,
568.     ROUND(AVG(f.skip_rate),4) AS avg_skip_rate,
569.     ROUND(AVG(f.listening_time),2) AS avg_listening_time,
570.     COUNT(*) AS total_plays
571. FROM Fact_Listening f
572. JOIN Dim_User u ON f.user_id = u.user_id
573. JOIN Dim_Track t ON f.track_id = t.track_id
574. GROUP BY u.is_churned
575. ''
576. result66 = run_query(q66)
577. display(result66)
```

Pengguna Active memiliki total pemutaran lebih dari pengguna *Churned* Perbedaan terlihat pada metrik *engagement* karena Pengguna *Churned* memiliki *avg_skip_rate* lebih tinggi.

Output Program 17. Churn vs Audio features

578.		<i>user_status</i>		<i>avg_energy</i>		<i>avg_danceability</i>		<i>avg_valence</i>		<i>avg_tempo</i>		<i>avg_skip_rate</i>
579.		Active		0.665		0.653		0.528		120.96		0.2979
				200.77		37108						
580.		Churned		0.666		0.653		0.527		121.18		0.3013
				198.52		12892						

5.1.7. Preferensi Genre *Playlist* Berdasarkan Kelompok Usia

Preferensi genre *playlist* berdasarkan kelompok umur pengguna akan menggunakan data yang dikelompokkan berdasarkan age group dan *playlist* genre, kemudian dihitung total pemutaran, rata-rata waktu dengar, dan *skip rate*. Analisis ini dapat digunakan sebagai dasar dalam strategi rekomendasi *playlist* yang lebih personal.

Kode Program 23. Genre playlist

```
581. q67 = ''
582. SELECT
583.     u.age_group,
584.     p.playlist_genre,
585.     COUNT(*)                AS total_plays,
586.     ROUND(AVG(f.listening_time),2) AS avg_listening_time,
587.     ROUND(AVG(f.skip_rate),4)    AS avg_skip_rate
588. FROM Fact_Listening f
589. JOIN Dim_User      u ON f.user_id      = u.user_id
590. JOIN Dim_Playlist p ON f.playlist_id = p.playlist_id
591. GROUP BY u.age_group, p.playlist_genre
592. ORDER BY u.age_group, total_plays DESC
593. ''
594. result67 = run_query(q67)
595. display(result67)
```

Kelompok umur 18–25 menunjukkan kecenderungan paling tinggi pada genre hip-hop, dengan total 1.182 *plays* yang menjadi pemutaran tertinggi pada kelompok umur tersebut. Genre rock memiliki rata-rata waktu dengar tertinggi di antara lima genre teratas, yaitu 205,02 detik. Kelompok umur <18 punya genre indie dan turkish, memiliki *skip rate* yang cukup tinggi.

Output Program 18. Genre playlist

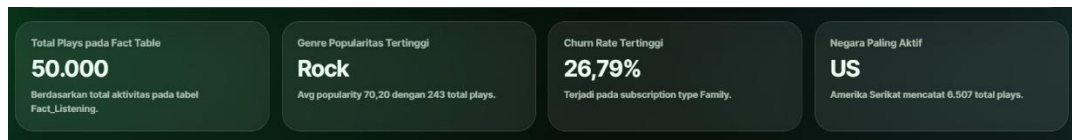
596.		age_group		playlist_genre		total_plays		avg_listening_time		avg_skip_rate	
597.		-----		-----		-----:		-----:		-----:	
598.		18-25		hip-hop		1182		201.82		0.2982	
599.		18-25		rock		909		205.02		0.2972	
600.		18-25		electronic		904		203.67		0.2940	
601.		18-25		pop		738		202.29		0.2984	
602.		18-25		latin		695		201.00		0.3069	
603.		...		...		...		...		...	
604.		<18		arabic		45		184.98		0.2869	
605.		<18		blues		44		202.64		0.2941	
606.		<18		indie		42		187.29		0.3576	
607.		<18		turkish		40		184.15		0.3465	
608.		<18		lofi		39		194.69		0.3028	

5.2 Visualisasi

Visualisasi data dilakukan untuk menyajikan hasil analisis *data warehouse* dalam bentuk tampilan *dashboard* yang lebih mudah dipahami. *Dashboard* dibuat berdasarkan hasil *query* yang telah dilakukan pada bagian sebelumnya. Tujuannya untuk membantu pengguna dalam membaca pola data secara lebih cepat.

5.2.1. Dashboard

Dashboard ditampilkan beberapa kartu ringkasan atau *key performance indicator* (KPI). Kartu ini berfungsi untuk memberikan gambaran umum mengenai kondisi data secara cepat sebelum pengguna melihat grafik yang lebih detail.\

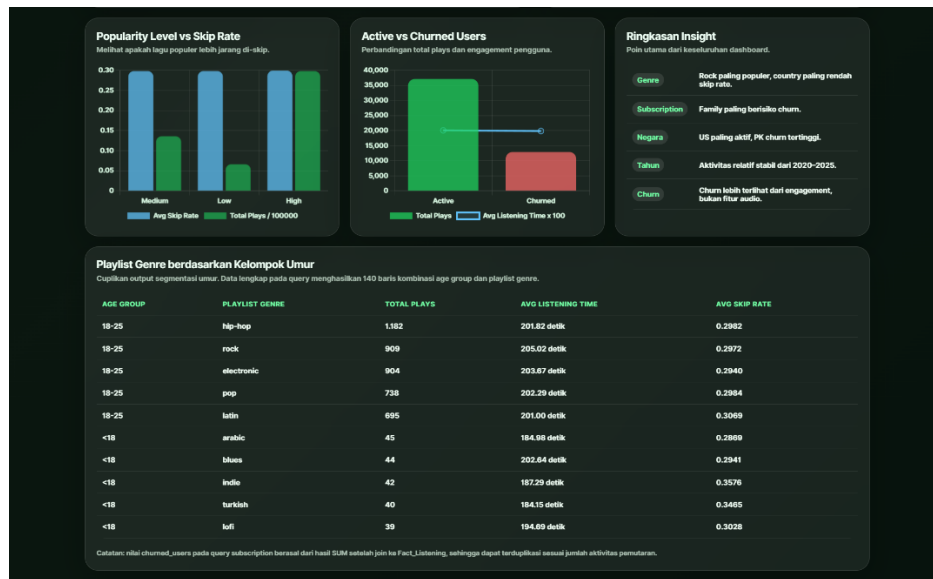


Gambar 5. 1 *Key performance indicator*

KPI yang ditampilkan pada *dashboard* terdiri dari total *plays*, genre dengan popularitas tertinggi, *churn* rate tertinggi, dan negara dengan aktivitas pengguna paling tinggi. Berdasarkan hasil visualisasi, jumlah total *plays* pada tabel fakta adalah sebanyak 50.000 data aktivitas pemutaran. Genre dengan rata-rata popularitas tertinggi adalah rock, dengan nilai rata-rata popularitas sebesar 70,20. *Churn* rate tertinggi terdapat pada jenis langganan Family, yaitu sebesar 26,79%. Sementara itu, negara dengan jumlah pemutaran tertinggi adalah United States (US/Amerika Serikat) dengan total 6.507 *plays*.



Gambar 5. 2 *Dashboard* (1)

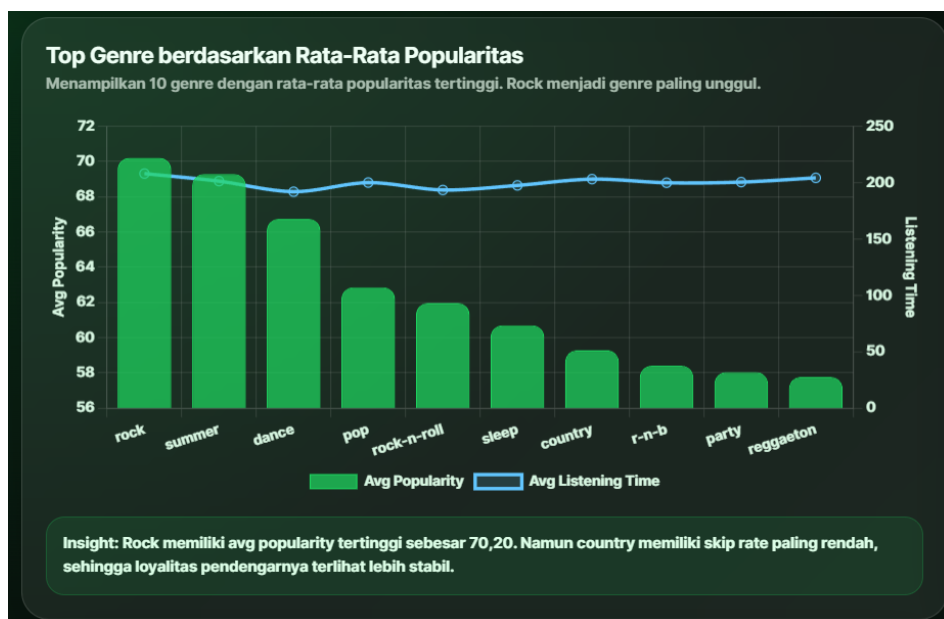


Gambar 5. 3 Dashboard (2)

Di atas adalah gambar keseluruhan dari *dashboard* yang sudah dibuat dalam bentuk *website* berbasis HTML. Visualnya diambil dari hasil visualisasi per-poin yang sudah dibuat pada Google Colab.

5.2.2. Genre Teratas Berdasarkan Rata-Rata Popularitas

Bagian ini menampilkan 10 genre musik dengan nilai rata-rata popularitas tertinggi. Grafik digunakan untuk melihat genre yang memiliki performa paling kuat pada data *Listening Behavior*.

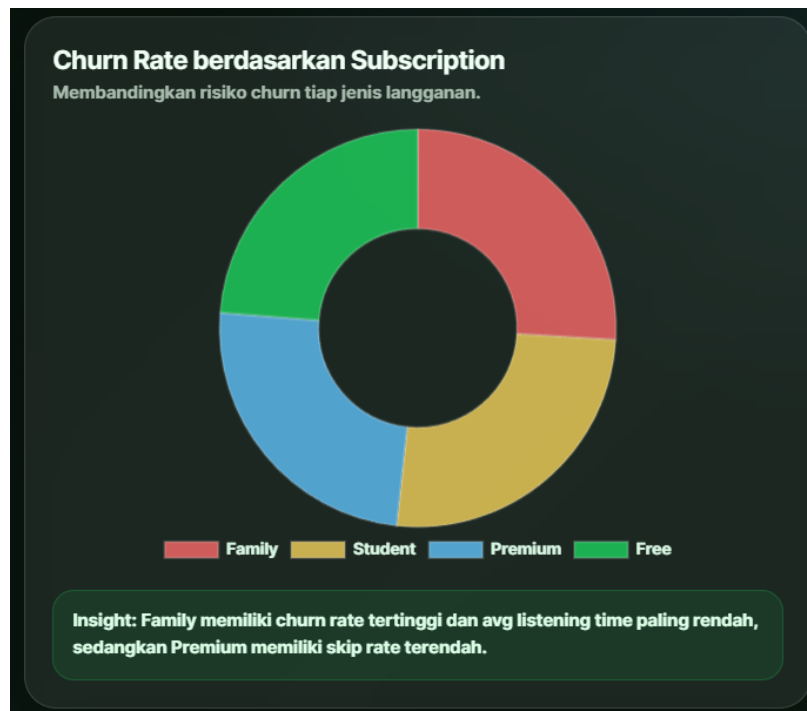


Gambar 5. 4 Top Genre

Genre rock berada di posisi tertinggi dengan rata-rata popularitas juga waktu dengar tertinggi. Hasil ini menunjukkan bahwa rock menjadi genre dengan daya tarik paling kuat dibandingkan genre lain yang dianalisis. Artinya, genre ini tidak hanya unggul dari segi popularitas, tetapi juga mampu mempertahankan durasi dengar pengguna lebih lama. Genre country memiliki *skip rate* paling rendah, yaitu 0,2807, sehingga dapat dikatakan memiliki loyalitas pendengar yang cukup stabil.

5.2.3. Churn Rate Berdasarkan Jenis Langganan

Bagian ini membandingkan tingkat *churn* pada setiap jenis langganan. Tujuannya untuk melihat kategori *subscription* yang memiliki risiko kehilangan pengguna paling besar. Kondisi di bawah menunjukkan bahwa pengguna Family memiliki *engagement* yang relatif lebih rendah dibandingkan kategori *subscription* lainnya.

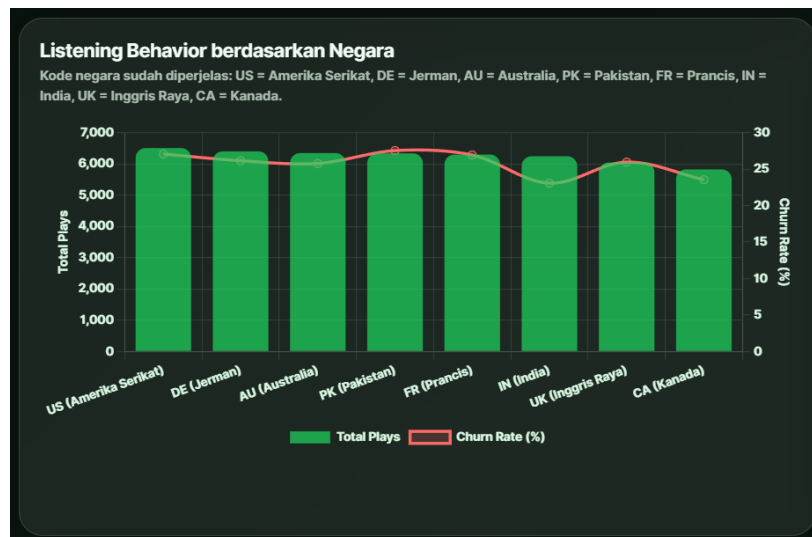


Gambar 5. 5 Churn Rate

Jenis langganan Premium menunjukkan performa *engagement* yang lebih baik. *Skip rate* pada kategori ini menjadi yang terendah, yaitu 0,2909, dengan rata-rata waktu dengar tertinggi sebesar 201,21 detik. Hal ini mengindikasikan bahwa pengguna Premium cenderung lebih stabil dalam menggunakan layanan.

5.2.4. *Listening Behavior* Berdasarkan Negara

Grafik ini menampilkan aktivitas mendengarkan musik dari tiap negara. Grafik membandingkan total *plays*, rata-rata waktu dengar, *skip rate*, rata-rata lagu per hari, dan *churn rate*.

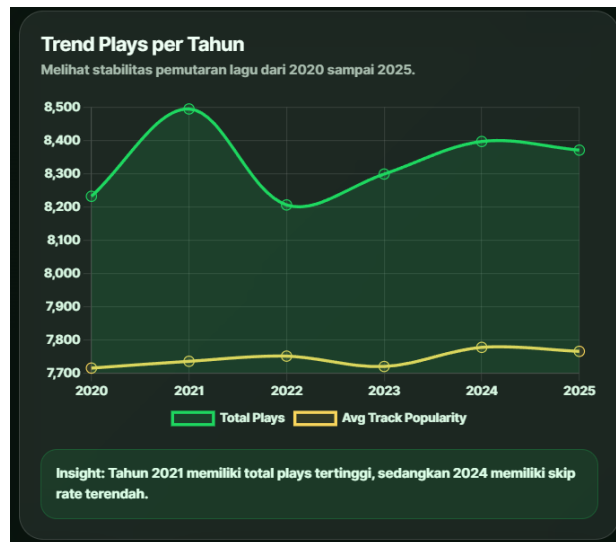


Gambar 5. 6 *Listening Behavior*

Jumlah menunjukkan bahwa Amerika Serikat menjadi negara dengan aktivitas pengguna paling tinggi. Pakistan memiliki *churn-rate* tertinggi sebesar 27,54% dan *skip rate* tertinggi sebesar 0,3030. Angka ini menunjukkan adanya potensi masalah *engagement* pada pengguna dari Pakistan, karena pengguna lebih sering melewati lagu dan memiliki risiko *churn* lebih besar.

5.2.5. *Trend Plays per Tahun*

Grafik ini membantu melihat apakah aktivitas pengguna mengalami kenaikan, penurunan, atau cenderung stabil dalam rentang waktu perkembangan jumlah pemutaran lagu dari tahun 2020 sampai 2025.

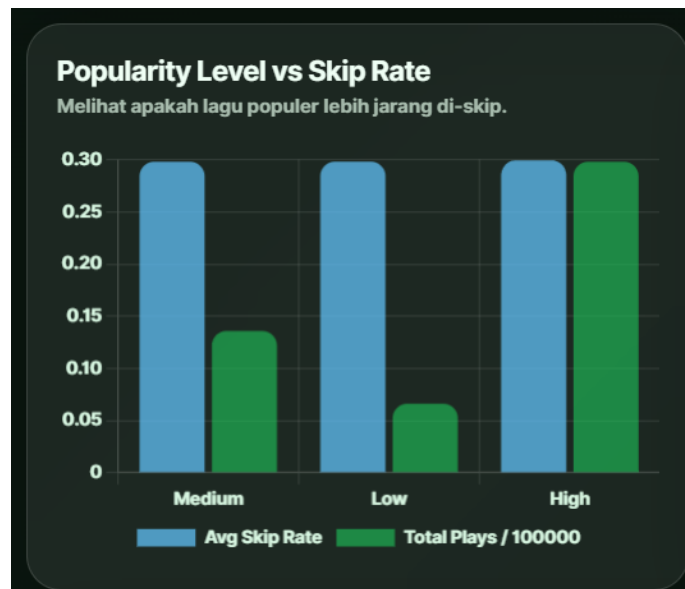


Gambar 5. 7 *Trend Plays*

Jumlah pemutaran lagu terlihat cukup stabil setiap tahun. Selisih antar tahun tidak terlalu besar, sehingga aktivitas mendengarkan musik dapat dikatakan konsisten.

5.2.6. *Popularity Level terhadap Skip rate*

Bagian ini membandingkan tingkat popularitas lagu dengan kecenderungan pengguna untuk melewati lagu. Kategori yang digunakan adalah High, Medium, dan Low. Kategori High memiliki total *plays* tertinggi tapi memiliki *skip rate* paling tinggi. Artinya, lagu yang populer belum tentu selalu didengarkan sampai lama oleh pengguna.

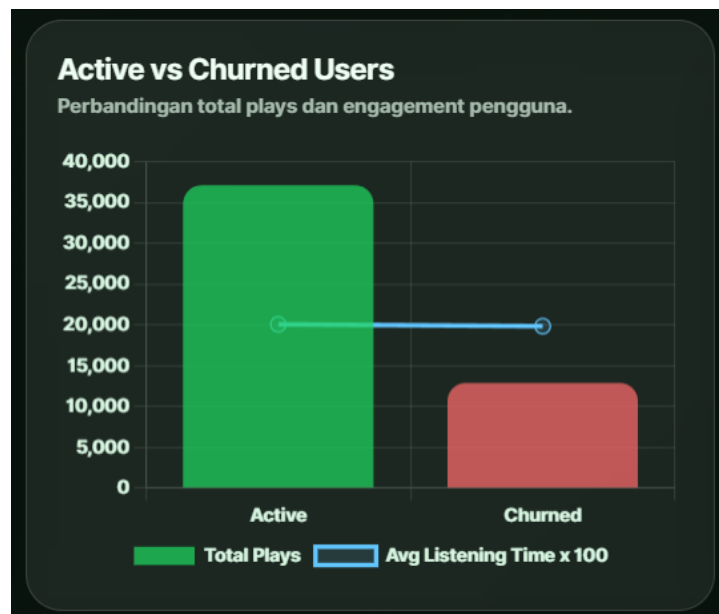


Gambar 5. 8 *Popularity Level terhadap Skip Rate*

Hasil ini menunjukkan bahwa lagu dengan popularitas menengah justru memiliki tingkat penerimaan yang sedikit lebih baik. Popularitas tinggi tetap penting, tetapi belum tentu menjadi satu-satunya indikator kenyamanan pengguna saat mendengarkan lagu.

5.2.7. Perbandingan Pengguna *Active* dan *Churned*

Bagian ini membandingkan pengguna yang masih aktif dengan pengguna yang sudah *churned*. Metrik yang digunakan meliputi total *plays*, rata-rata waktu dengar, *skip rate*, dan beberapa *audio features*. Perbedaan di grafik menunjukkan bahwa pengguna aktif memiliki aktivitas mendengarkan musik yang jauh lebih tinggi. Rata-rata waktu dengar pengguna *Active* mencapai 200,77 detik, sedangkan pengguna *Churned* hanya 198,52 detik. *Skip rate* pengguna *Churned* juga lebih tinggi, yaitu 0,3013, dibandingkan pengguna *Active* sebesar 0,2979.



Gambar 5. 9 Perbandingan Pengguna *Active* dan *Churned*

Nilai *audio features* antara dua kelompok ini relatif mirip. *Energy*, *danceability*, *valence*, dan tempo tidak menunjukkan perbedaan besar. Jadi, *churn* lebih terlihat dari rendahnya *engagement* pengguna, bukan dari jenis atau karakteristik audio lagu yang mereka dengarkan.

5.2.8. *Playlist Genre* Berdasarkan Kelompok Umur

Bagian ini memperlihatkan preferensi genre *playlist* untuk setiap kelompok umur. Data ini membantu kita melihat genre mana yang paling banyak diputar oleh kelompok umur tertentu.

AGE GROUP	PLAYLIST GENRE	TOTAL PLAYS	AVG LISTENING TIME	AVG SKIP RATE
18-25	hip-hop	1.182	201.82 detik	0.2982
18-25	rock	909	205.02 detik	0.2972
18-25	electronic	904	203.67 detik	0.2940
18-25	pop	738	202.29 detik	0.2984
18-25	latin	695	201.00 detik	0.3069
<18	arabic	45	184.98 detik	0.2869
<18	blues	44	202.64 detik	0.2941
<18	indie	42	187.29 detik	0.3576
<18	turkish	40	184.15 detik	0.3465
<18	lofi	39	194.69 detik	0.3028

Gambar 5. 10 *Playlist Genre Berdasarkan Kelompok Umur*

Genre *electronic* juga cukup kuat dengan rata-rata waktu dengar 203,67 detik dan *skip rate* rendah sebesar 0,2940. Walaupun hip-hop menjadi genre dengan jumlah pemutaran tertinggi, rock dan electronic terlihat lebih kuat dari durasi dengarnya.

5.3 Insight

Hasil analisis menunjukkan beberapa pola menarik dari perilaku pengguna Spotify, yaitu sebagai berikut.

- 1) Genre rock menjadi genre dengan rata-rata popularitas tertinggi. Genre ini juga memiliki waktu dengar yang cukup tinggi, sehingga bisa dilihat sebagai genre yang tidak hanya populer, tetapi juga mampu mempertahankan perhatian pendengar.
- 2) Jenis langganan Family memiliki *churn* rate tertinggi, yaitu 26,79%. Paket ini juga memiliki *skip rate* paling tinggi dan waktu dengar yang lebih rendah. Hal ini menunjukkan adanya kemungkinan bahwa pengguna Family kurang aktif atau kurang terikat dengan layanan dibandingkan jenis langganan lainnya.
- 3) Negara dengan total *plays* tertinggi adalah United States (US/Amerika Serikat) dengan 6.507 *plays*. Namun, negara dengan *churn* rate tertinggi adalah Pakistan (PK) sebesar 27,54%. Berarti banyaknya jumlah pemutaran tidak selalu berarti pengguna lebih loyal atau memiliki *engagement* yang lebih baik.
- 4) Kategori *Popularity Level* Medium memiliki *skip rate* terendah, yaitu 0,2979. Ini menjadi temuan yang cukup menarik karena lagu dengan *Popularity Level* High justru memiliki *skip rate* lebih tinggi. Artinya, lagu yang sangat populer belum tentu selalu menjadi lagu yang paling nyaman atau paling sesuai dengan preferensi pengguna.

- 5) Pengguna *Churned* memiliki rata-rata *skip rate* sebesar 0,3013 dan rata-rata *listening time* sebesar 198,52 detik. Pola ini menunjukkan bahwa pengguna yang *churn* cenderung lebih sering melewati lagu dan memiliki durasi dengar yang lebih pendek. Tanda-tanda *churn* dapat terlihat dari menurunnya *engagement* pengguna.
- 6) Kombinasi kelompok umur dan *playlist* genre teratas adalah 36–50 & hip-hop. Temuan ini menunjukkan bahwa genre hip-hop tidak hanya dominan pada kelompok usia muda, tetapi juga cukup kuat pada kelompok umur dewasa. Informasi ini dapat digunakan untuk melihat preferensi musik berdasarkan segmentasi umur pengguna.

BAB VI

PENUTUP

6.1 Kesimpulan

Data warehouse untuk analisis perilaku pengguna Spotify telah dirancang dengan menggunakan pendekatan *star schema*. Struktur *data warehouse* terdiri dari tabel fakta Fact_Listening dan beberapa tabel dimensi, yaitu Dim_User, Dim_Track, Dim_Playlist, dan Dim_Time melalui ETL lalu disimpan ke dalam *database*, yang awalnya merupakan tiga *dataset* mentah dari *platform* Kaggle.

Hasil analisis menunjukkan beberapa pola penting. Genre rock menjadi genre dengan rata-rata popularitas tertinggi. Jenis langganan Family memiliki *churn rate* tertinggi sebesar 26,79%, sehingga menjadi kategori yang paling berisiko mengalami kehilangan pengguna. Negara dengan total *plays* tertinggi adalah United States (US/Amerika Serikat) dengan 6.507 *plays*, sedangkan negara dengan *churn rate* tertinggi adalah Pakistan (PK) sebesar 27,54%. Lagu dengan *Popularity Level* Medium memiliki *skip rate* terendah, yaitu 0,2979. Artinya, lagu yang sangat populer belum tentu selalu menjadi lagu yang paling jarang dilewati pengguna. Pada perbandingan pengguna Active dan *Churned*, pengguna *Churned* memiliki *skip rate* lebih tinggi dan rata-rata waktu dengar lebih rendah. Hal ini menunjukkan bahwa *churn* lebih terlihat dari rendahnya *engagement* pengguna.

Dashboard yang dibuat membantu menampilkan hasil analisis dengan lebih mudah dipahami. Informasi yang sebelumnya berupa tabel dapat dilihat dalam bentuk KPI, grafik, dan tabel ringkasan. Terakhir, *data warehouse* tidak hanya menjadi tempat penyimpanan data, tetapi juga dapat digunakan untuk menemukan pola perilaku pengguna Spotify.

6.2 Saran

Pengembangan selanjutnya dapat dilakukan dengan menggunakan data Spotify yang lebih besar dan lebih aktual agar hasil analisis menjadi lebih representatif. Struktur *data warehouse* dapat diperluas dengan menambahkan dimensi baru, seperti dimensi perangkat, dimensi *engagement*, atau dimensi lokasi yang lebih detail.

LAMPIRAN

Berikut beberapa lampiran yang bisa dilihat sebagai pendukung pemahaman pembaca:

Collab Workspace:

<https://drive.google.com/file/d/1Y-6BFXDKCow11MJaKHHlsmObZ10pHrpz/view?usp=sharing>

Github:

<https://github.com/syzota/spotify-intelligence>

Website *Dashboard*:

<https://syzota.github.io/spotify-intelligence/>

Dataset Source (Kaggle):

<https://www.kaggle.com/datasets/nabihazahid/spotify-dataset-for-churn-analysis>

<https://www.kaggle.com/datasets/ambaliyagati/spotify-dataset-for-playing-around-with-sql>

<https://www.kaggle.com/datasets/solomonameh/spotify-music-dataset>

HTML Source Code:

Kode Program 24. *Dashboard* Source Code

```
1.  <!DOCTYPE html>
2.  <html lang="id">
3.  <head>
4.    <meta charset="UTF-8" />
5.    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6.    <title>Dashboard Data warehouse Spotify</title>
7.    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
8.    <style>
9.      :root {
10.        --bg: #07130d;
11.        --panel: rgba(255,255,255,0.075);
12.        --panel-2: rgba(255,255,255,0.115);
13.        --line: rgba(255,255,255,0.14);
14.        --text: #ffffff5;
15.        --muted: #a8bcae;
16.        --green: #1ed760;
17.        --green-2: #73ffa6;
18.        --yellow: #f9d65c;
19.        --red: #ff6b6b;
20.        --blue: #62c6ff;
21.      }
22.      * { box-sizing: border-box; }
```

```

23.     body {
24.         margin: 0;
25.         font-family: Inter, ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont,
           "Segoe UI", sans-serif;
26.         background:
27.             radial-gradient(circle at top left, rgba(30,215,96,.28), transparent 32%),
28.             radial-gradient(circle at top right, rgba(98,198,255,.16), transparent 25%),
29.             linear-gradient(135deg, #06110b 0%, #0b1710 50%, #07130d 100%);
30.         color: var(--text);
31.         min-height: 100vh;
32.     }
33.     .wrap { width: min(1400px, 94vw); margin: 0 auto; padding: 34px 0 48px; }
34.     header {
35.         display: flex; justify-content: space-between; align-items: flex-start; gap: 20px;
36.         padding: 26px; border: 1px solid var(--line); border-radius: 28px;
37.         background: linear-gradient(135deg, rgba(255,255,255,.12),
           rgba(255,255,255,.04));
38.         box-shadow: 0 20px 70px rgba(0,0,0,.35);
39.         backdrop-filter: blur(16px);
40.     }
41.     .eyebrow { color: var(--green-2); font-weight: 800; letter-spacing: .12em; font-
           size: 12px; text-transform: uppercase; }
42.     h1 { margin: 10px 0 8px; font-size: clamp(30px, 4vw, 56px); line-height: 1; letter-
           spacing: -0.05em; }
43.     .subtitle { color: var(--muted); max-width: 720px; font-size: 15px; line-height:
           1.65; }
44.     .badge {
45.         display: inline-flex; align-items: center; gap: 10px; padding: 12px 14px; border-
           radius: 999px;
46.         background: rgba(30,215,96,.14); color: var(--green-2); border: 1px solid
           rgba(30,215,96,.35);
47.         white-space: nowrap; font-weight: 700;
48.     }
49.     .spotify-dot { width: 12px; height: 12px; border-radius: 999px; background: var(--
           green); box-shadow: 0 0 18px var(--green); }
50.     .kpis { display: grid; grid-template-columns: repeat(4, 1fr); gap: 16px; margin:
           18px 0; }
51.     .card {
52.         background: var(--panel); border: 1px solid var(--line); border-radius: 24px;
53.         padding: 20px; box-shadow: 0 18px 55px rgba(0,0,0,.22); backdrop-filter:
           blur(16px);
54.     }
55.     .kpi-label { color: var(--muted); font-size: 13px; margin-bottom: 8px; }
56.     .kpi-value { font-size: 30px; font-weight: 900; letter-spacing: -0.04em; }
57.     .kpi-note { color: var(--muted); font-size: 12px; margin-top: 8px; line-height: 1.5;
           }
58.     .grid { display: grid; grid-template-columns: 1.15fr .85fr; gap: 18px; margin-top:
           18px; }

```

```

59.     .grid-3 { display: grid; grid-template-columns: repeat(3, 1fr); gap: 18px; margin-
top: 18px; }
60.     .card h2 { margin: 0 0 4px; font-size: 19px; letter-spacing: -0.02em; }
61.     .desc { color: var(--muted); font-size: 13px; line-height: 1.55; margin-bottom:
16px; }
62.     .chart-box { position: relative; height: 330px; }
63.     .chart-box.sm { height: 285px; }
64.     .insight {
65.         margin-top: 14px; padding: 13px 14px; border-radius: 16px; background:
rgba(30,215,96,.10);
66.         border: 1px solid rgba(30,215,96,.22); color: #ddffe8; font-size: 13px; line-
height: 1.55;
67.     }
68.     table { width: 100%; border-collapse: collapse; overflow: hidden; border-radius:
16px; font-size: 13px; }
69.     th, td { padding: 12px 10px; border-bottom: 1px solid rgba(255,255,255,.09); text-
align: left; }
70.     th { color: var(--green-2); font-size: 12px; text-transform: uppercase; letter-
spacing: .06em; }
71.     td { color: #f2fff6; }
72.     tr:hover td { background: rgba(255,255,255,.055); }
73.     .pill { display: inline-block; padding: 5px 9px; border-radius: 999px; background:
rgba(255,255,255,.10); color: var(--green-2); font-weight: 700; }
74.     .footer-note { color: var(--muted); margin-top: 18px; font-size: 12px; line-height:
1.6; }
75.     @media (max-width: 1000px) {
76.         .kpis, .grid-3, .grid { grid-template-columns: 1fr; }
77.         header { flex-direction: column; }
78.     }
79. </style>
80. </head>
81. <body>
82.     <div class="wrap">
83.         <header>
84.             <div>
85.                 <div class="eyebrow">Spotify Data warehouse Dashboard</div>
86.                 <h1>Dashboard Analisis Listening Behavior</h1>
87.                 <div class="subtitle">
88.                     Dashboard ini merangkum hasil query data warehouse Spotify: popularitas genre,
churn berdasarkan langganan,
89.                     perilaku pengguna berdasarkan negara, tren tahunan, fitur audio, serta
perbandingan pengguna aktif dan churned.
90.                 </div>
91.             </div>
92.             <div class="badge"><span class="spotify-dot"></span> Star schema Analytics</div>
93.         </header>
94.
95.         <section class="kpis">
96.             <div class="card">
97.                 <div class="kpi-label">Total Plays pada Fact table</div>

```

```

98.         <div class="kpi-value">50.000</div>
99.         <div class="kpi-note">Berdasarkan total aktivitas pada tabel
Fact_Listening.</div>
100.      </div>
101.      <div class="card">
102.          <div class="kpi-label">Genre Popularitas Tertinggi</div>
103.          <div class="kpi-value">Rock</div>
104.          <div class="kpi-note">Avg popularity 70,20 dengan 243 total plays.</div>
105.      </div>
106.      <div class="card">
107.          <div class="kpi-label">Churn Rate Tertinggi</div>
108.          <div class="kpi-value">26,79%</div>
109.          <div class="kpi-note">Terjadi pada subscription type Family.</div>
110.      </div>
111.      <div class="card">
112.          <div class="kpi-label">Negara Paling Aktif</div>
113.          <div class="kpi-value">US</div>
114.          <div class="kpi-note">Amerika Serikat mencatat 6.507 total plays.</div>
115.      </div>
116.  </section>
117.
118.  <section class="grid">
119.      <div class="card">
120.          <h2>Genre Teratas berdasarkan Rata-Rata Popularitas</h2>
121.          <div class="desc">Menampilkan 10 genre dengan rata-rata popularitas tertinggi.
Rock menjadi genre paling unggul.</div>
122.          <div class="chart-box"><canvas id="genreChart"></canvas></div>
123.          <div class="insight">Insight: Rock memiliki avg popularity tertinggi sebesar
70,20. Namun country memiliki skip rate paling rendah, sehingga loyalitas pendengarnya
terlihat lebih stabil.</div>
124.      </div>
125.      <div class="card">
126.          <h2>Churn Rate berdasarkan Subscription</h2>
127.          <div class="desc">Membandingkan risiko churn tiap jenis langganan.</div>
128.          <div class="chart-box"><canvas id="subscriptionChart"></canvas></div>
129.          <div class="insight">Insight: Family memiliki churn rate tertinggi dan avg
listening time paling rendah, sedangkan Premium memiliki skip rate terendah.</div>
130.      </div>
131.  </section>
132.
133.  <section class="grid">
134.      <div class="card">
135.          <h2>Listening Behavior berdasarkan Negara</h2>
136.          <div class="desc">Kode negara sudah diperjelas: US = Amerika Serikat, DE =
German, AU = Australia, PK = Pakistan, FR = Prancis, IN = India, UK = Inggris Raya, CA
= Kanada.</div>
137.          <div class="chart-box"><canvas id="countryChart"></canvas></div>
138.      </div>
139.      <div class="card">
140.          <h2>Trend Plays per Tahun</h2>

```

```
141.         <div class="desc">Melihat stabilitas pemutaran lagu dari 2020 sampai 2025.</div>
142.         <div class="chart-box"><canvas id="yearlyChart"></canvas></div>
143.         <div class="insight">Insight: Tahun 2021 memiliki total plays tertinggi,
sedangkan 2024 memiliki skip rate terendah.</div>
144.     </div>
145. </section>
146.
147. <section class="grid-3">
148.     <div class="card">
149.         <h2>Popularity Level vs Skip rate</h2>
150.         <div class="desc">Melihat apakah lagu populer lebih jarang di-skip.</div>
151.         <div class="chart-box sm"><canvas id="audioChart"></canvas></div>
152.     </div>
153.     <div class="card">
154.         <h2>Active vs Churned Users</h2>
155.         <div class="desc">Perbandingan total plays dan engagement pengguna.</div>
156.         <div class="chart-box sm"><canvas id="churnAudioChart"></canvas></div>
157.     </div>
158.     <div class="card">
159.         <h2>Ringkasan Insight</h2>
160.         <div class="desc">Poin utama dari keseluruhan dashboard.</div>
161.         <table>
162.             <tbody>
163.                 <tr><td><span class="pill">Genre</span></td><td>Rock paling populer, country
paling rendah skip rate.</td></tr>
164.                 <tr><td><span class="pill">Subscription</span></td><td>Family paling
berisiko churn.</td></tr>
165.                 <tr><td><span class="pill">Negara</span></td><td>US paling aktif, PK churn
tertinggi.</td></tr>
166.                 <tr><td><span class="pill">Tahun</span></td><td>Aktivitas relatif stabil
dari 2020-2025.</td></tr>
167.                 <tr><td><span class="pill">Churn</span></td><td>Churn lebih terlihat dari
engagement, bukan fitur audio.</td></tr>
168.             </tbody>
169.         </table>
170.     </div>
171. </section>
172.
173. <section class="card" style="margin-top:18px;">
174.     <h2>Playlist Genre berdasarkan Kelompok Umur</h2>
175.     <div class="desc">Cuplikan output segmentasi umur. Data lengkap pada query
menghasilkan 140 baris kombinasi age group dan playlist genre.</div>
176.     <table id="ageTable">
177.         <thead>
178.             <tr>
179.                 <th>Age Group</th>
180.                 <th>Playlist Genre</th>
181.                 <th>Total Plays</th>
182.                 <th>Avg Listening time</th>
183.                 <th>Avg Skip rate</th>
```

```

184.         </tr>
185.     </thead>
186.     <tbody></tbody>
187. </table>
188. <div class="footer-note">
189.     Catatan: nilai churned_users pada query subscription berasal dari hasil SUM
        setelah join ke Fact_Listening, sehingga dapat terduplikasi sesuai jumlah aktivitas
        pemutaran.
190. </div>
191. </section>
192. </div>
193.
194. <script>
195. const data = {"top_genre": [{"genre": "rock", "total_plays": 243, "avg_popularity":
        70.2, "avg_skip_rate": 0.302, "avg_listening_time": 207.98}, {"genre": "summer",
        "total_plays": 234, "avg_popularity": 69.29, "avg_skip_rate": 0.3031,
        "avg_listening_time": 201.29}, {"genre": "dance", "total_plays": 185, "avg_popularity":
        66.74, "avg_skip_rate": 0.313, "avg_listening_time": 191.91}, {"genre": "pop",
        "total_plays": 116, "avg_popularity": 62.85, "avg_skip_rate": 0.3329,
        "avg_listening_time": 200.03}, {"genre": "rock-n-roll", "total_plays": 149,
        "avg_popularity": 61.96, "avg_skip_rate": 0.2866, "avg_listening_time": 193.42},
        {"genre": "sleep", "total_plays": 138, "avg_popularity": 60.69, "avg_skip_rate": 0.3128,
        "avg_listening_time": 197.51}, {"genre": "country", "total_plays": 113,
        "avg_popularity": 59.28, "avg_skip_rate": 0.2807, "avg_listening_time": 203.1},
        {"genre": "r-n-b", "total_plays": 224, "avg_popularity": 58.4, "avg_skip_rate": 0.3002,
        "avg_listening_time": 199.86}, {"genre": "party", "total_plays": 115, "avg_popularity":
        58.03, "avg_skip_rate": 0.3092, "avg_listening_time": 200.44}, {"genre": "reggaeton",
        "total_plays": 164, "avg_popularity": 57.76, "avg_skip_rate": 0.3137,
        "avg_listening_time": 204.2}], "subscription": [{"subscription_type": "Family",
        "total_users": 1795, "churned_users": 3135, "churn_rate_pct": 26.79, "avg_skip_rate":
        0.3028, "avg_listening_time": 197.87}, {"subscription_type": "Student", "total_users":
        1828, "churned_users": 3274, "churn_rate_pct": 26.6, "avg_skip_rate": 0.302,
        "avg_listening_time": 201.07}, {"subscription_type": "Premium", "total_users": 1992,
        "churned_users": 3374, "churn_rate_pct": 25.33, "avg_skip_rate": 0.2909,
        "avg_listening_time": 201.21}, {"subscription_type": "Free", "total_users": 1917,
        "churned_users": 3109, "churn_rate_pct": 24.54, "avg_skip_rate": 0.3001,
        "avg_listening_time": 200.41}], "countries": [{"country": "US", "country_name":
        "Amerika Serikat", "total_plays": 6507, "avg_listening_time": 199.19, "avg_skip_rate":
        0.2945, "avg_songs_per_day": 48.86, "churn_rate_pct": 27.06}, {"country": "DE",
        "country_name": "Jerman", "total_plays": 6401, "avg_listening_time": 203.48,
        "avg_skip_rate": 0.2998, "avg_songs_per_day": 47.46, "churn_rate_pct": 26.15},
        {"country": "AU", "country_name": "Australia", "total_plays": 6346,
        "avg_listening_time": 198.13, "avg_skip_rate": 0.3, "avg_songs_per_day": 49.56,
        "churn_rate_pct": 25.76}, {"country": "PK", "country_name": "Pakistan", "total_plays":
        6337, "avg_listening_time": 199.95, "avg_skip_rate": 0.303, "avg_songs_per_day": 51.36,
        "churn_rate_pct": 27.54}, {"country": "FR", "country_name": "Prancis", "total_plays":
        6298, "avg_listening_time": 201.24, "avg_skip_rate": 0.2956, "avg_songs_per_day":
        50.85, "churn_rate_pct": 26.93}, {"country": "IN", "country_name": "India",
        "total_plays": 6245, "avg_listening_time": 201.94, "avg_skip_rate": 0.2984,
        "avg_songs_per_day": 49.94, "churn_rate_pct": 23.07}, {"country": "UK", "country_name":

```

```

"Inggris Raya", "total_plays": 6043, "avg_listening_time": 198.05, "avg_skip_rate":
0.2976, "avg_songs_per_day": 49.11, "churn_rate_pct": 25.96}, {"country": "CA",
"country_name": "Kanada", "total_plays": 5823, "avg_listening_time": 199.4,
"avg_skip_rate": 0.3014, "avg_songs_per_day": 52.59, "churn_rate_pct": 23.54}],
"audio_skip": [{"popularity_level": "Medium", "avg_energy": 0.647, "avg_danceability":
0.663, "avg_tempo": 120.87, "avg_valence": 0.533, "avg_skip_rate": 0.2979,
"total_plays": 13590}, {"popularity_level": "Low", "avg_energy": null,
"avg_danceability": null, "avg_tempo": null, "avg_valence": null, "avg_skip_rate":
0.298, "total_plays": 6623}, {"popularity_level": "High", "avg_energy": 0.671,
"avg_danceability": 0.65, "avg_tempo": 121.06, "avg_valence": 0.526, "avg_skip_rate":
0.2993, "total_plays": 29787}], "yearly": [{"year": 2020, "total_plays": 8232,
"avg_listening_time": 201.56, "avg_track_popularity": 64.29, "avg_skip_rate": 0.299},
{"year": 2021, "total_plays": 8495, "avg_listening_time": 199.64,
"avg_track_popularity": 64.46, "avg_skip_rate": 0.301}, {"year": 2022, "total_plays":
8206, "avg_listening_time": 201.71, "avg_track_popularity": 64.59, "avg_skip_rate":
0.2973}, {"year": 2023, "total_plays": 8299, "avg_listening_time": 198.24,
"avg_track_popularity": 64.33, "avg_skip_rate": 0.2988}, {"year": 2024, "total_plays":
8397, "avg_listening_time": 199.96, "avg_track_popularity": 64.81, "avg_skip_rate":
0.2972}, {"year": 2025, "total_plays": 8371, "avg_listening_time": 200.08,
"avg_track_popularity": 64.71, "avg_skip_rate": 0.2993}], "churn_audio":
[{"user_status": "Active", "avg_energy": 0.665, "avg_danceability": 0.653,
"avg_valence": 0.528, "avg_tempo": 120.96, "avg_skip_rate": 0.2979,
"avg_listening_time": 200.77, "total_plays": 37108}, {"user_status": "Churned",
"avg_energy": 0.666, "avg_danceability": 0.653, "avg_valence": 0.527, "avg_tempo":
121.18, "avg_skip_rate": 0.3013, "avg_listening_time": 198.52, "total_plays": 12892}],
"age_playlist_sample": [{"age_group": "18-25", "playlist_genre": "hip-hop",
"total_plays": 1182, "avg_listening_time": 201.82, "avg_skip_rate": 0.2982},
{"age_group": "18-25", "playlist_genre": "rock", "total_plays": 909,
"avg_listening_time": 205.02, "avg_skip_rate": 0.2972}, {"age_group": "18-25",
"playlist_genre": "electronic", "total_plays": 904, "avg_listening_time": 203.67,
"avg_skip_rate": 0.294}, {"age_group": "18-25", "playlist_genre": "pop", "total_plays":
738, "avg_listening_time": 202.29, "avg_skip_rate": 0.2984}, {"age_group": "18-25",
"playlist_genre": "latin", "total_plays": 695, "avg_listening_time": 201.0,
"avg_skip_rate": 0.3069}, {"age_group": "<18", "playlist_genre": "arabic",
"total_plays": 45, "avg_listening_time": 184.98, "avg_skip_rate": 0.2869},
{"age_group": "<18", "playlist_genre": "blues", "total_plays": 44,
"avg_listening_time": 202.64, "avg_skip_rate": 0.2941}, {"age_group": "<18",
"playlist_genre": "indie", "total_plays": 42, "avg_listening_time": 187.29,
"avg_skip_rate": 0.3576}, {"age_group": "<18", "playlist_genre": "turkish",
"total_plays": 40, "avg_listening_time": 184.15, "avg_skip_rate": 0.3465},
{"age_group": "<18", "playlist_genre": "lofi", "total_plays": 39, "avg_listening_time":
194.69, "avg_skip_rate": 0.3028}]];

```

196.

197. Chart.defaults.color = '#d9f8e3';

198. Chart.defaults.borderColor = 'rgba(255,255,255,.10)';

199. Chart.defaults.font.family = 'Inter, system-ui, sans-serif';

200.

201. function cssVar(name) {

202. return getComputedStyle(document.documentElement).getPropertyValue(name).trim();

203. }


```

204.
205. const green = cssVar('--green');
206. const green2 = cssVar('--green-2');
207. const yellow = cssVar('--yellow');
208. const red = cssVar('--red');
209. const blue = cssVar('--blue');
210.
211. new Chart(document.getElementById('genreChart'), {
212.   type: 'bar',
213.   data: {
214.     labels: data.top_genre.map(d => d.genre),
215.     datasets: [
216.       {
217.         label: 'Avg Popularity',
218.         data: data.top_genre.map(d => d.avg_popularity),
219.         backgroundColor: 'rgba(30, 215, 96, .72)',
220.         borderColor: green,
221.         borderWidth: 1,
222.         borderRadius: 10
223.       },
224.       {
225.         label: 'Avg Listening time',
226.         data: data.top_genre.map(d => d.avg_listening_time),
227.         type: 'line',
228.         yAxisID: 'y1',
229.         borderColor: blue,
230.         backgroundColor: 'rgba(98,198,255,.15)',
231.         tension: .35,
232.         pointRadius: 4
233.       }
234.     ]
235.   },
236.   options: {
237.     responsive: true, maintainAspectRatio: false,
238.     scales: {
239.       y: { beginAtZero: false, title: {display:true, text:'Avg Popularity'} },
240.       y1: { position:'right', grid:{drawOnChartArea:false}, title: {display:true,
241.         text:'Listening time'} }
242.     },
243.     plugins: { legend: { position: 'bottom' } }
244.   });
245.
246. new Chart(document.getElementById('subscriptionChart'), {
247.   type: 'doughnut',
248.   data: {
249.     labels: data.subscription.map(d => d.subscription_type),
250.     datasets: [{
251.       data: data.subscription.map(d => d.churn_rate_pct),

```

```

252.         backgroundColor:    ['rgba(255,107,107,.78)',    'rgba(249,214,92,.78)',
    'rgba(98,198,255,.78)', 'rgba(30,215,96,.78)'],
253.         borderColor: 'rgba(255,255,255,.16)',
254.         borderWidth: 2
255.     }]
256. },
257.     options: {
258.         responsive: true, maintainAspectRatio: false,
259.         plugins: {
260.             legend: { position: 'bottom' },
261.             tooltip: { callbacks: { label: ctx => `${ctx.label}: ${ctx.raw}%` } }
262.         }
263.     }
264. });
265.
266. new Chart(document.getElementById('countryChart'), {
267.     type: 'bar',
268.     data: {
269.         labels: data.countries.map(d => `${d.country} (${d.country_name})`),
270.         datasets: [
271.             {
272.                 label: 'Total Plays',
273.                 data: data.countries.map(d => d.total_plays),
274.                 backgroundColor: 'rgba(30, 215, 96, .70)',
275.                 borderRadius: 10
276.             },
277.             {
278.                 label: 'Churn Rate (%)',
279.                 data: data.countries.map(d => d.churn_rate_pct),
280.                 type: 'line',
281.                 yAxisID: 'y1',
282.                 borderColor: red,
283.                 backgroundColor: 'rgba(255,107,107,.18)',
284.                 tension: .35,
285.                 pointRadius: 4
286.             }
287.         ]
288.     },
289.     options: {
290.         responsive: true, maintainAspectRatio: false,
291.         scales: {
292.             x: { ticks: { maxRotation: 30, minRotation: 20 } },
293.             y: { title: {display:true, text:'Total Plays'} },
294.             y1: { position:'right', grid:{drawOnChartArea:false}, title: {display:true,
    text:'Churn Rate (%)'} }
295.         },
296.         plugins: { legend: { position: 'bottom' } }
297.     }
298. });
299.

```

```

300. new Chart(document.getElementById('yearlyChart'), {
301.     type: 'line',
302.     data: {
303.         labels: data.yearly.map(d => d.year),
304.         datasets: [
305.             {
306.                 label: 'Total Plays',
307.                 data: data.yearly.map(d => d.total_plays),
308.                 borderColor: green,
309.                 backgroundColor: 'rgba(30,215,96,.14)',
310.                 fill: true,
311.                 tension: .35,
312.                 pointRadius: 5
313.             },
314.             {
315.                 label: 'Avg Track Popularity',
316.                 data: data.yearly.map(d => d.avg_track_popularity * 120),
317.                 borderColor: yellow,
318.                 backgroundColor: 'rgba(249,214,92,.10)',
319.                 tension: .35,
320.                 pointRadius: 5
321.             }
322.         ]
323.     },
324.     options: {
325.         responsive: true, maintainAspectRatio: false,
326.         plugins: {
327.             legend: { position: 'bottom' },
328.             tooltip: {
329.                 callbacks: {
330.                     label: function(ctx) {
331.                         if (ctx.dataset.label === 'Avg Track Popularity') return `Avg Track
Popularity: ${ (ctx.raw/120).toFixed(2) }`;
332.                         return `${ctx.dataset.label}: ${ctx.raw}`;
333.                     }
334.                 }
335.             }
336.         }
337.     }
338. });
339.
340. new Chart(document.getElementById('audioChart'), {
341.     type: 'bar',
342.     data: {
343.         labels: data.audio_skip.map(d => d.popularity_level),
344.         datasets: [
345.             {
346.                 label: 'Avg Skip rate',
347.                 data: data.audio_skip.map(d => d.avg_skip_rate),
348.                 backgroundColor: 'rgba(98,198,255,.72)',

```

```

349.         borderRadius: 10
350.     },
351.     {
352.         label: 'Total Plays / 100000',
353.         data: data.audio_skip.map(d => d.total_plays / 100000),
354.         backgroundColor: 'rgba(30,215,96,.55)',
355.         borderRadius: 10
356.     }
357. ]
358. },
359. options: {
360.     responsive: true, maintainAspectRatio: false,
361.     scales: { y: { beginAtZero: true } },
362.     plugins: { legend: { position:'bottom' } }
363. }
364. });
365.
366. new Chart(document.getElementById('churnAudioChart'), {
367.     type: 'bar',
368.     data: {
369.         labels: data.churn_audio.map(d => d.user_status),
370.         datasets: [
371.             {
372.                 label: 'Total Plays',
373.                 data: data.churn_audio.map(d => d.total_plays),
374.                 backgroundColor: ['rgba(30,215,96,.72)', 'rgba(255,107,107,.72)'],
375.                 borderRadius: 12
376.             },
377.             {
378.                 label: 'Avg Listening time x 100',
379.                 data: data.churn_audio.map(d => d.avg_listening_time * 100),
380.                 type: 'line',
381.                 borderColor: blue,
382.                 backgroundColor: 'rgba(98,198,255,.12)',
383.                 tension: .25,
384.                 pointRadius: 5
385.             }
386.         ]
387.     },
388.     options: {
389.         responsive: true, maintainAspectRatio: false,
390.         plugins: {
391.             legend: { position:'bottom' },
392.             tooltip: {
393.                 callbacks: {
394.                     label: function(ctx) {
395.                         if (ctx.dataset.label === 'Avg Listening time x 100') return `Avg Listening
time: ${((ctx.raw/100).toFixed(2))} detik`;
396.                         return `${ctx.dataset.label}: ${ctx.raw}`;
397.                     }

```

```
398.         }
399.     }
400. }
401. }
402. });
403.
404. const tbody = document.querySelector('#ageTable tbody');
405. data.age_playlist_sample.forEach(row => {
406.     const tr = document.createElement('tr');
407.     tr.innerHTML = `
408.         <td>${row.age_group}</td>
409.         <td>${row.playlist_genre}</td>
410.         <td>${row.total_plays.toLocaleString('id-ID')}</td>
411.         <td>${row.avg_listening_time.toFixed(2)} detik</td>
412.         <td>${row.avg_skip_rate.toFixed(4)}</td>
413.     `;
414.     tbody.appendChild(tr);
415. });
416. </script>
417. </body>
418. </html>
419.
420.
```

DAFTAR PUSTAKA

- Ahlstrand, J., & Borg, A. (2025). *Using Transformers for B2B Contractual Churn Prediction Based on Customer Behavior Data*. *I(Iceis)*, 562–571.
<https://doi.org/10.5220/0013432500003929>
- Cheng, H., & He, J. (2025). *Advanced customer churn prediction for a music streaming digital marketing service using attention graph-based deep learning approach*.
- Inmon, W. H. (2002). *Building the Data warehouse*.
- Khutami, J. Q., Adinda, N., Asmara, P., Auliana, L., & Raharja, J. (2024). *PENGARUH ARTIFICIAL INTELLIGENCE TERHADAP CUSTOMER (Survey pada Pengguna Aplikasi Spotify di Jatinangor)*. *13*(2), 510–517.
- Kimball, R., & Ross, M. (2002). *The Data warehouse Toolkit*.
- Turban, E., McLean, E. R., & Wetherbe, J. (2017). *Information Technology for Management: Transforming Organizations in the Digital Economy*. John Wiley & Sons.
- Connolly, T. M., & Begg, C. E. (2015). *Database Systems: A Practical Approach to Design, Implementation, and Management* (6th ed.). Boston, MA: Pearson Education.
- Inmon, W. H. (2005). *Building the Data warehouse* (4th ed.). Indianapolis, IN: John Wiley & Sons.
- Kimball, R., & Ross, M. (2013). *The Data warehouse Toolkit: The Definitive Guide to Dimensional Modeling* (3rd ed.). Indianapolis, IN: John Wiley & Sons.